

```
from google.colab import drive
drive.mount('/content/drive/')
```

↗ Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", force_remount=True).

```
#Step 1: Import the required modules
```

```
import math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from math import sqrt
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
from sklearn.metrics import mean_absolute_percentage_error
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import warnings
warnings.filterwarnings("ignore")
```

```
from statsmodels.tsa.arima.model import ARIMA
```

```
from datetime import timedelta
```

```
from prophet import Prophet
```

```
from sklearn import preprocessing
```

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier, export_graphviz, DecisionTreeRegressor
import seaborn as sns
sns.set(style="white")
sns.set(style="whitegrid", color_codes=True)
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_squared_error as MSE
```

```
#Step 2: Generate the dataset
```

```
data=pd.read_csv('/content/drive/MyDrive/Sales.csv')
```

```
data
```



	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)
0	2023-01-09	1	10613	442	0	0.00	15
1	2023-09-11	32	320011	2377	0	0.00	1
2	2023-07-24	18	180107	7476	2	687.94	15
3	2024-12-09	42	422609	7983	4	1031.04	29
4	2023-05-22	40	401005	7268	6	684.50	849
...	...	...	...	...	...	...	...
1242783	2024-01-01	64	640607	1016	0	0.00	4
1242784	2023-11-06	34	340906	7181	0	0.00	11
1242785	2025-02-17	62	620014	5928	1	387.07	33
1242786	2024-08-19	7	71525	7860	0	0.00	2
1242787	2024-08-12	11	110120	1228	2	512.51	88



1242788 rows x 7 columns

data.dtypes



	0
Date	object
Department	int64
Category	int64
Brand	int64
Sales (Units)	int64
Sales (Amount)	float64
Inventory (Units)	int64

dtype: object

data.isnull().sum()

```
↗ 0
```

	0
Date	0
Department	0
Category	0
Brand	0
Sales (Units)	0
Sales (Amount)	0
Inventory (Units)	0

```
dtype: int64
```

```
data['Department'].nunique()
```

```
↗ 56
```

```
data['Brand'].nunique()
```

```
↗ 2323
```

```
data['Category'].nunique()
```

```
↗ 2763
```

```
data['Date'].nunique()
```

```
↗ 126
```

```
pd.set_option('display.float_format', '{:.2f}'.format)
data.describe()
```



	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)
count	1242788.00	1242788.00	1242788.00	1242788.00	1242788.00	1242788.00
mean	23.96	240024.62	3949.67	4.15	3697.91	79.32
std	17.12	171152.66	2698.67	17.76	32858.05	263.41
min	1.00	10001.00	4.00	-43.00	-251721.64	-4958.00
25%	11.00	110108.00	1145.00	0.00	0.00	5.00
50%	21.00	210103.00	3838.00	1.00	178.44	18.00
75%	36.00	361818.00	6542.00	3.00	1612.70	65.00
max	66.00	660504.00	8219.00	3262.00	13197240.68	17871.00




```
int((data['Sales (Units)']<0).sum())
```

 4105

```
int((data['Sales (Amount)']<0).sum())
```

 5747

```
int((data['Inventory (Units)']<0).sum())
```

 23410

```
data['Date'] =pd.to_datetime(data['Date'])
```

```
data= data.sort_values('Date')
```

```
data.groupby(['Department','Category','Brand']).size().sort_values(ascending=False)
```



0

Department	Category	Brand	
1	10001	3525	126
9	90220	836	126
		2844	126
	90222	944	126
		2129	126
...	...	...	...
6	60105	8186	1
31	310204	5993	1
41	411007	878	1
9	90536	7133	1
12	120206	7888	1

17680 rows x 1 columns

dtype: int64

data



	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)
38160	2023-01-02	32	320009	1275	1	503.53	0
312659	2023-01-02	66	660003	2489	1	31.68	64
635912	2023-01-02	18	180016	78	9	524.90	139
312764	2023-01-02	29	290710	6897	3	1911.20	3
635806	2023-01-02	14	140525	3911	1	447.41	0
...	...	...	...	...	...	...	...
319465	2025-05-26	47	470215	78	0	0.00	18
319454	2025-05-26	26	260206	6807	6	360.38	51
1087328	2025-05-26	40	400102	8061	3	516.93	20
319221	2025-05-26	2	20347	7765	0	0.00	3
113186	2025-05-26	27	270205	6289	17	6825.15	263

1242788 rows x 7 columns



Assumptions: There are no product returns from customers.

The company does not issue any refunds after a sale.

The sales system does not allow the sale of a product when its stock is zero or insufficient.

```
# Remove rows with negative values in the target columns.
```

```
data_t = data[
    (data['Sales (Units)'] >= 0) &
    (data['Sales (Amount)'] >= 0) &
    (data['Inventory (Units)'] >= 0)
]
```

```
data_t['Product_Id'] = ('P_' +
    data_t['Department'].astype(str) + '|' +
    data_t['Category'].astype(str) + '|' +
    data_t['Brand'].astype(str)
)
```

data_t

	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id
38160	2023-01-02	32	320009	1275	1	503.53	0	P_32 320009 1275
312659	2023-01-02	66	660003	2489	1	31.68	64	P_66 660003 2489
635912	2023-01-02	18	180016	78	9	524.90	139	P_18 180016 78
312764	2023-01-02	29	290710	6897	3	1911.20	3	P_29 290710 6897
635806	2023-01-02	14	140525	3911	1	447.41	0	P_14 140525 3911
...	...	...	...	...	...	...	...	...
319465	2025-05-26	47	470215	78	0	0.00	18	P_47 470215 78
319454	2025-05-26	26	260206	6807	6	360.38	51	P_26 260206 6807
1087328	2025-05-26	40	400102	8061	3	516.93	20	P_40 400102 8061
319221	2025-05-26	2	20347	7765	0	0.00	3	P_2 20347 7765
113186	2025-05-26	27	270205	6289	17	6825.15	263	P_27 270205 6289

1213863 rows x 8 columns

```
data_t.groupby('Product_Id').size().sort_values(ascending=False)
```



0

Product_Id

P_10I100701I6347	126
P_9I90529I6758	126
P_9I90529I6098	126
P_9I90528I6760	126
P_9I90512I6760	126
...	...
P_21I210103I7197	1
P_40I400105I5337	1
P_41I410101I690	1
P_40I400010I1572	1
P_12I120408I606	1

17457 rows × 1 columns

dtype: int64

data_t[(data_t['Department']==3) & (data_t['Category']==30103) & (data_t['Brand']==614)]



	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id
855690	2023-01-02	3	30103	614	450	503104.65	11392	P_3I30103I614
141320	2023-01-09	3	30103	614	268	295734.14	11581	P_3I30103I614
567067	2023-01-16	3	30103	614	283	313575.25	11475	P_3I30103I614
1122792	2023-01-23	3	30103	614	282	305816.38	11209	P_3I30103I614
910153	2023-01-30	3	30103	614	283	307676.09	11505	P_3I30103I614
...	...	...	...	...	...	...	...	...
1229399	2025-04-28	3	30103	614	468	428696.90	12317	P_3I30103I614
568852	2025-05-05	3	30103	614	329	352932.11	12012	P_3I30103I614
974477	2025-05-12	3	30103	614	522	461302.50	11961	P_3I30103I614
1231892	2025-05-19	3	30103	614	427	412211.87	11854	P_3I30103I614
995558	2025-05-26	3	30103	614	788	587822.52	11119	P_3I30103I614

126 rows × 8 columns

```
produits_par_dept = data_t.groupby('Department')['Product_Id'].nunique().rename('Nb_Produits')
ventes_par_dept = data_t.groupby('Department')['Sales (Units)'].sum().rename('Ventes_Total')
```

```
classement_departements = (
    pd.concat([produits_par_dept, ventes_par_dept], axis=1)
    .sort_values(by=['Nb_Produits', 'Ventes_Total'], ascending=False)
    .reset_index()
)
```

```
classement_departements['Classement'] = classement_departements.index
```

```
ordre_depts = classement_departements.set_index('Department')['Classement'].to_dict()
```

```
data_t['Classement_Dept'] = data_t['Department'].map(ordre_depts)
```

```
data1_trie = data_t.sort_values(by='Classement_Dept').drop(columns='Classement_Dept')
```

data1_trie

	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id
672156	2023-02-13	11	110403	4187	0	0.00	24	P_11I110403I4187
1172327	2024-06-10	11	110127	5676	0	0.00	6	P_11I110127I5676
18608	2023-09-25	11	110404	7497	0	0.00	9	P_11I110404I7497
517267	2025-03-24	11	110119	4144	0	0.00	9	P_11I110119I4144
1201187	2023-02-13	11	110107	7454	0	0.00	1	P_11I110107I7454
...	...	...	...	...	...	...	...	...
577894	2023-05-29	65	650001	7411	0	0.00	1	P_65I650001I7411
925374	2024-01-29	65	650003	7411	0	0.00	1	P_65I650003I7411
895468	2024-01-29	65	650002	7411	0	0.00	1	P_65I650002I7411
803176	2024-09-09	65	650001	7411	0	0.00	1	P_65I650001I7411
1147471	2024-01-29	65	650001	7411	0	0.00	1	P_65I650001I7411

1213863 rows × 8 columns

```
def is_difficult_series(group):
    sales = group['Sales (Units)']
    return (
        len(sales) < 104 or
```

```

    sales.std() == 0    or
    #sales.autocorr(lag=1) < 0.3 or
    (sales == 0).sum() / len(sales) > 0.6
)

produits_difficiles = (
    data_t.groupby(['Department', 'Category', 'Brand'])
    .filter(is_difficult_series)
)

data1 = (
    data_t.groupby(['Department', 'Category', 'Brand'])
    .filter(lambda g: not is_difficult_series(g))
)

```

data1



	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id	Classement_Dept
635912	2023-01-02	18	180016	78	9	524.90	139	P_18I180016I78	6
312764	2023-01-02	29	290710	6897	3	1911.20	3	P_29I290710I6897	13
1090510	2023-01-02	26	260307	2166	4	5513.79	48	P_26I260307I2166	22
312872	2023-01-02	20	200611	5038	7	2860.86	48	P_20I200611I5038	11
635940	2023-01-02	3	30207	6935	2	2351.72	96	P_3I30207I6935	14
...	...	...	...	...	...	...	...	...	...
80629	2025-05-26	46	460001	5977	5	84.51	0	P_46I460001I5977	42
319799	2025-05-26	2	20121	6516	0	0.00	18	P_2I20121I6516	3
319505	2025-05-26	15	150025	575	0	0.00	7	P_15I150025I575	34
319221	2025-05-26	2	20347	7765	0	0.00	3	P_2I20347I7765	3
113186	2025-05-26	27	270205	6289	17	6825.15	263	P_27I270205I6289	8

496786 rows × 9 columns

```
data1.groupby('Product_Id').size().sort_values(ascending=False)
```



0

Product_Id

P_9I90222I5875	126
P_11I110102I1228	126
P_9I90681I3792	126
P_9I90222I4490	126
P_11I110102I5676	126
...	...
P_47I470402I1016	104
P_45I450401I7797	104
P_46I460001I6251	104
P_11I110114I4340	104
P_45I450201I7206	104

4070 rows × 1 columns

dtype: int64

data1['Department'].nunique()



53

departments_data1 = set(data_t['Department'].unique())

departments_sans_difficiles = set(data1['Department'].unique())

departments_exclus = departments_data1 - departments_sans_difficiles

```
print("Départements exclus car tous leurs produits sont difficiles :")
print(departments_exclus)
```



```
Départements exclus car tous leurs produits sont difficiles :
{np.int64(24), np.int64(65), np.int64(63)}
```

data1



	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id	Classement_Dept	
635912	2023-01-02	18	180016	78	9	524.90	139	P_18 180016 78	6	
312764	2023-01-02	29	290710	6897	3	1911.20	3	P_29 290710 6897	13	
1090510	2023-01-02	26	260307	2166	4	5513.79	48	P_26 260307 2166	22	
312872	2023-01-02	20	200611	5038	7	2860.86	48	P_20 200611 5038	11	
635940	2023-01-02	3	30207	6935	2	2351.72	96	P_3 30207 6935	14	
...	...	...	...	...	...	...	...	...	...	
80629	2025-05-26	46	460001	5977	5	84.51	0	P_46 460001 5977	42	
319799	2025-05-26	2	20121	6516	0	0.00	18	P_2 20121 6516	3	
319505	2025-05-26	15	150025	575	0	0.00	7	P_15 150025 575	34	
319221	2025-05-26	2	20347	7765	0	0.00	3	P_2 20347 7765	3	
113186	2025-05-26	27	270205	6289	17	6825.15	263	P_27 270205 6289	8	

496786 rows × 9 columns

```
stats_data1 = data1.groupby('Product_Id')['Sales (Units)'].describe()
```

```
stats_data1.head(50)
```



	count	mean	std	min	25%	50%	75%	max	
Product_Id									
P_10I100701I3678	123.00	4.29	4.55	0.00	2.00	3.00	5.00	29.00	
P_10I100701I5123	123.00	2.71	3.37	0.00	1.00	2.00	3.00	21.00	
P_10I100701I6347	126.00	7.38	5.76	0.00	3.00	5.00	11.00	30.00	
P_10I100701I952	125.00	3.91	3.82	0.00	1.00	3.00	5.00	21.00	
P_10I100702I2478	112.00	1.33	1.57	0.00	0.00	1.00	2.00	8.00	
P_10I100702I6347	116.00	8.86	5.50	1.00	5.00	8.00	11.00	26.00	
P_10I100703I6347	108.00	1.44	1.42	0.00	0.00	1.00	2.00	6.00	
P_11I110101I5676	115.00	0.71	1.09	0.00	0.00	0.00	1.00	6.00	
P_11I110102I1228	126.00	4.40	4.51	0.00	2.00	3.00	6.00	32.00	
P_11I110102I1973	108.00	1.02	1.48	0.00	0.00	0.50	1.00	7.00	
P_11I110102I3982	106.00	2.51	2.25	0.00	1.00	2.00	3.00	11.00	
P_11I110102I4340	125.00	3.70	2.80	0.00	2.00	3.00	5.00	12.00	
P_11I110102I5333	123.00	1.39	1.68	0.00	0.00	1.00	2.00	10.00	
P_11I110102I5676	126.00	6.06	4.24	0.00	3.00	5.00	8.00	25.00	
P_11I110102I6066	125.00	5.45	5.74	0.00	1.00	4.00	7.00	33.00	
P_11I110102I6289	126.00	9.79	8.06	0.00	4.00	7.00	14.00	40.00	
P_11I110102I6364	126.00	1.30	1.97	0.00	0.00	1.00	2.00	11.00	
P_11I110102I6725	125.00	4.38	3.48	0.00	2.00	4.00	6.00	16.00	
P_11I110103I1228	116.00	0.92	1.15	0.00	0.00	1.00	1.00	5.00	
P_11I110103I1229	125.00	3.06	4.93	0.00	0.00	2.00	3.00	31.00	
P_11I110103I3982	108.00	4.56	4.98	0.00	1.00	3.00	6.00	32.00	
P_11I110103I5676	125.00	3.90	4.73	0.00	0.00	3.00	6.00	27.00	
P_11I110103I6066	126.00	11.28	9.31	1.00	5.00	8.00	14.00	45.00	
P_11I110103I6289	125.00	7.42	10.11	0.00	2.00	5.00	8.00	72.00	
P_11I110103I6364	123.00	1.39	1.50	0.00	0.00	1.00	2.00	8.00	
P_11I110103I6476	126.00	2.56	3.02	0.00	0.00	2.00	4.00	18.00	
P_11I110103I6725	126.00	8.06	15.08	0.00	2.00	4.00	9.00	137.00	
P_11I110103I7467	125.00	1.94	2.10	0.00	0.00	1.00	3.00	10.00	
P_11I110104I1483	112.00	1.04	2.04	0.00	0.00	0.00	1.00	14.00	

P_11111010415333	109.00	1.25	1.59	0.00	0.00	1.00	2.00	7.00
P_11111010511228	105.00	2.68	3.10	0.00	0.00	2.00	4.00	19.00
P_11111010515676	123.00	5.07	4.96	0.00	1.50	3.00	7.00	26.00
P_11111010516066	123.00	2.46	2.62	0.00	1.00	2.00	4.00	13.00
P_11111010516364	126.00	1.01	1.42	0.00	0.00	0.00	2.00	7.00
P_11111010516476	108.00	0.71	1.00	0.00	0.00	0.00	1.00	5.00
P_11111010516725	125.00	6.32	6.24	0.00	2.00	6.00	8.00	48.00
P_11111010711483	115.00	8.17	9.75	0.00	2.00	4.00	11.00	59.00
P_11111010711973	114.00	12.31	11.75	0.00	2.00	9.00	19.00	42.00
P_11111010714303	126.00	32.34	27.25	0.00	12.25	27.50	49.50	119.00
P_1111101071571	108.00	2.56	2.31	0.00	1.00	2.00	4.00	11.00
P_11111010716066	126.00	7.80	12.54	0.00	1.00	3.00	9.00	71.00
P_1111101071614	126.00	51.69	30.86	14.00	28.00	42.00	68.00	169.00
P_11111010716200	126.00	125.94	129.15	0.00	25.00	72.50	222.50	534.00
P_11111010716289	125.00	23.88	18.16	0.00	9.00	22.00	33.00	72.00
P_11111010716364	123.00	2.67	3.14	0.00	0.00	2.00	4.00	17.00
P_11111010716476	126.00	4.88	5.70	0.00	1.00	3.00	7.00	31.00
P_11111010716725	115.00	2.72	3.59	0.00	0.00	1.00	4.00	21.00
P_11111010717308	107.00	3.15	3.09	0.00	0.00	3.00	5.00	17.00
P_11111010717309	107.00	0.98	1.05	0.00	0.00	1.00	2.00	4.00
P_11111010811228	126.00	6.98	7.08	0.00	2.25	4.00	8.75	40.00

Étapes suivantes : [Générer du code avec stats_data1](#) [Afficher les graphiques recommandés](#) [New interactive sheet](#)

```
data1[(data1['Department']==11)]
```

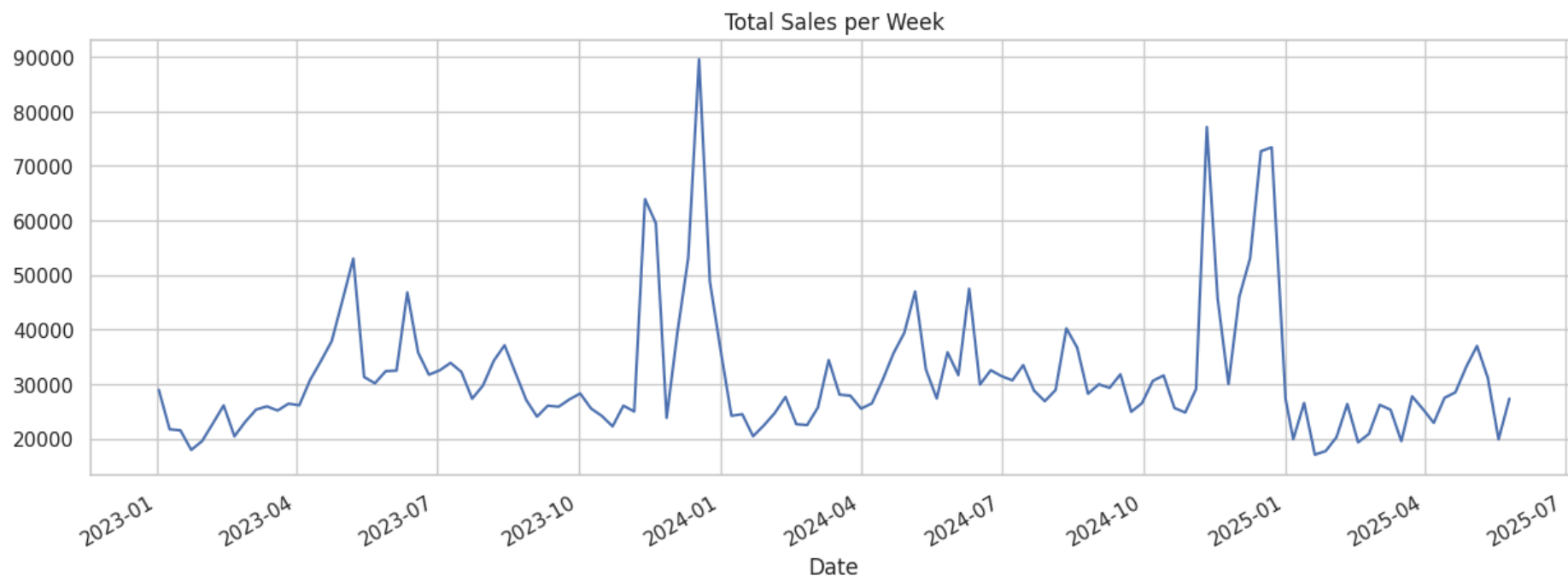


	Date	Department	Category	Brand	Sales (Units)	Sales (Amount)	Inventory (Units)	Product_Id	Classement_Dept	
9605	2023-01-02	11	110119	6066	15	7133.69	230	P_11 110119 6066	0	
638614	2023-01-02	11	110112	3982	1	542.50	0	P_11 110112 3982	0	
310054	2023-01-02	11	110119	6476	13	3508.79	137	P_11 110119 6476	0	
321486	2023-01-02	11	110108	614	1	430.17	96	P_11 110108 614	0	
321763	2023-01-02	11	110404	4187	6	2472.68	83	P_11 110404 4187	0	
...	...	...	...	...	...	...	...	...	...	
324371	2025-05-26	11	110119	614	12	10609.28	223	P_11 110119 614	0	
665474	2025-05-26	11	110107	4303	1	0.01	11	P_11 110107 4303	0	
1090214	2025-05-26	11	110104	5333	0	0.00	6	P_11 110104 5333	0	
675663	2025-05-26	11	110114	3982	0	0.00	0	P_11 110114 3982	0	
1086871	2025-05-26	11	110108	3982	2	0.02	4	P_11 110108 3982	0	

22134 rows x 9 columns

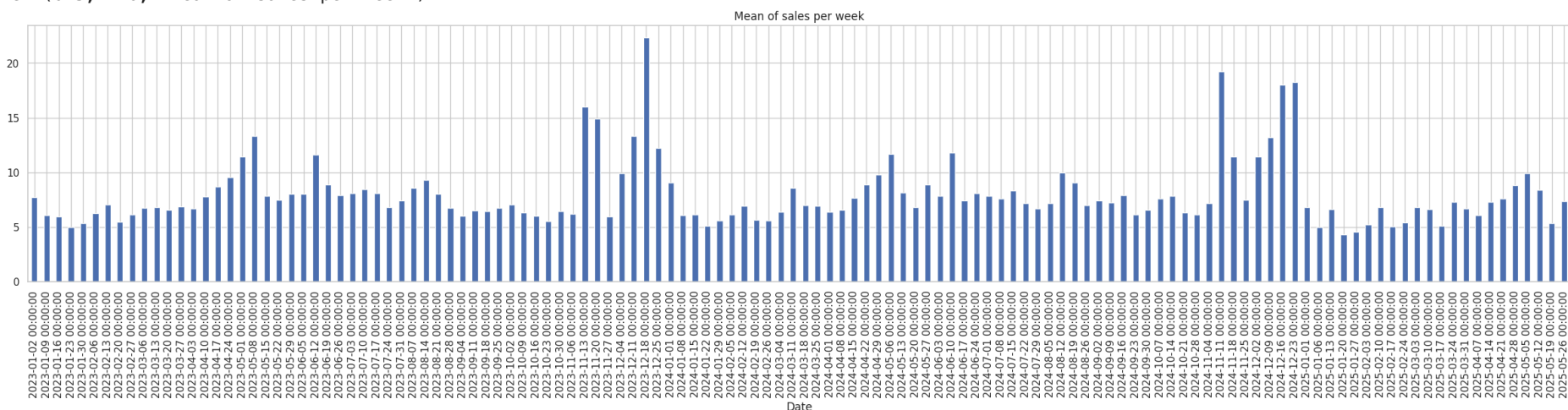
```
data1.groupby("Date")["Sales (Units)"].sum().plot(figsize=(15, 5))
plt.title("Total Sales per Week")
```

Text(0.5, 1.0, 'Total Sales per Week')



```
data1.groupby("Date")["Sales (Units)"].mean().plot(kind="bar", figsize=(30, 5))
plt.title("Mean of sales per week")
```

Text(0.5, 1.0, 'Mean of sales per week')



```
data1= data1.reset_index()
```

```
data1['Date'] =pd.to_datetime(data1['Date'])
```

```
data1= data1.sort_values('Date')
```

```
data1
```

</

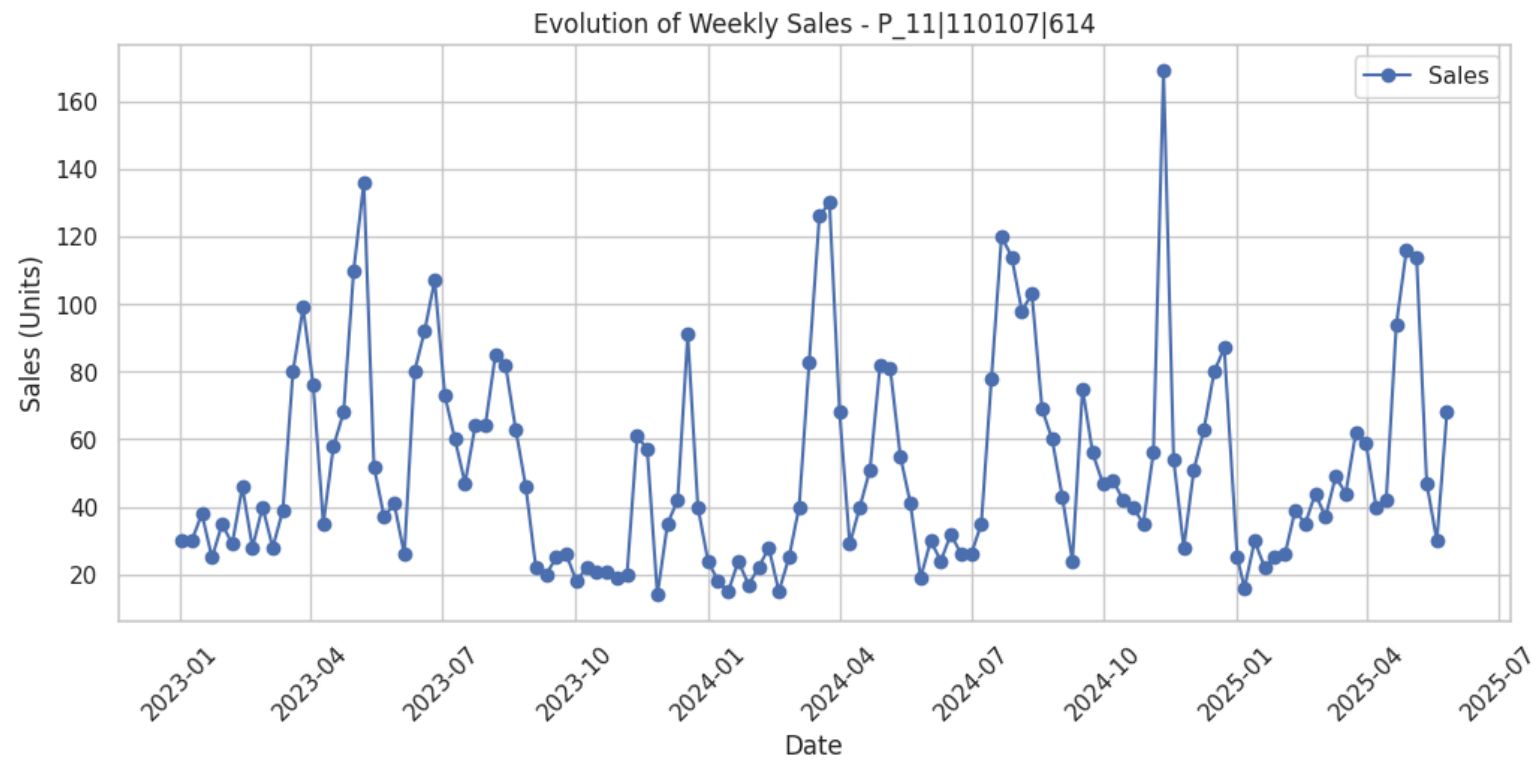
496786 rows x 10 columns

```
def Graph_sale(Product_Id):
    df = data1[data1['Product_Id'] == Product_Id].copy()
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date')

    plt.figure(figsize=(12, 5))
    plt.plot(df['Date'], df['Sales (Units)'], marker='o', label='Sales')
    plt.title(f"Evolution of Weekly Sales - {Product_Id}")
    plt.xlabel("Date")
    plt.ylabel("Sales (Units)")
    plt.grid(True)
    plt.legend()
    plt.xticks(rotation=45)
    plt.show()

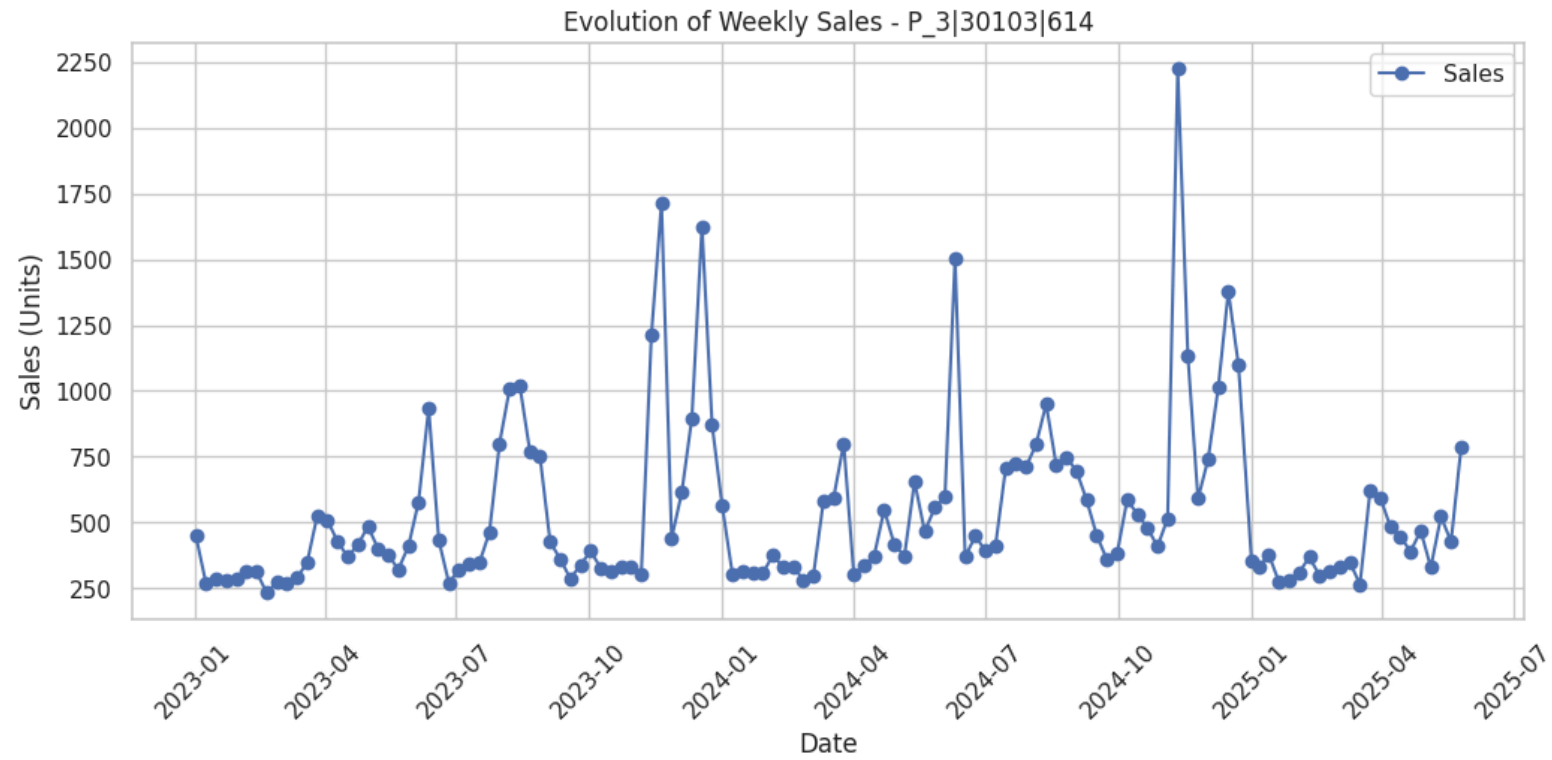
    print(df['Sales (Units)'].autocorr(1))
```

```
Graph_sale('P_11|110107|614')
```



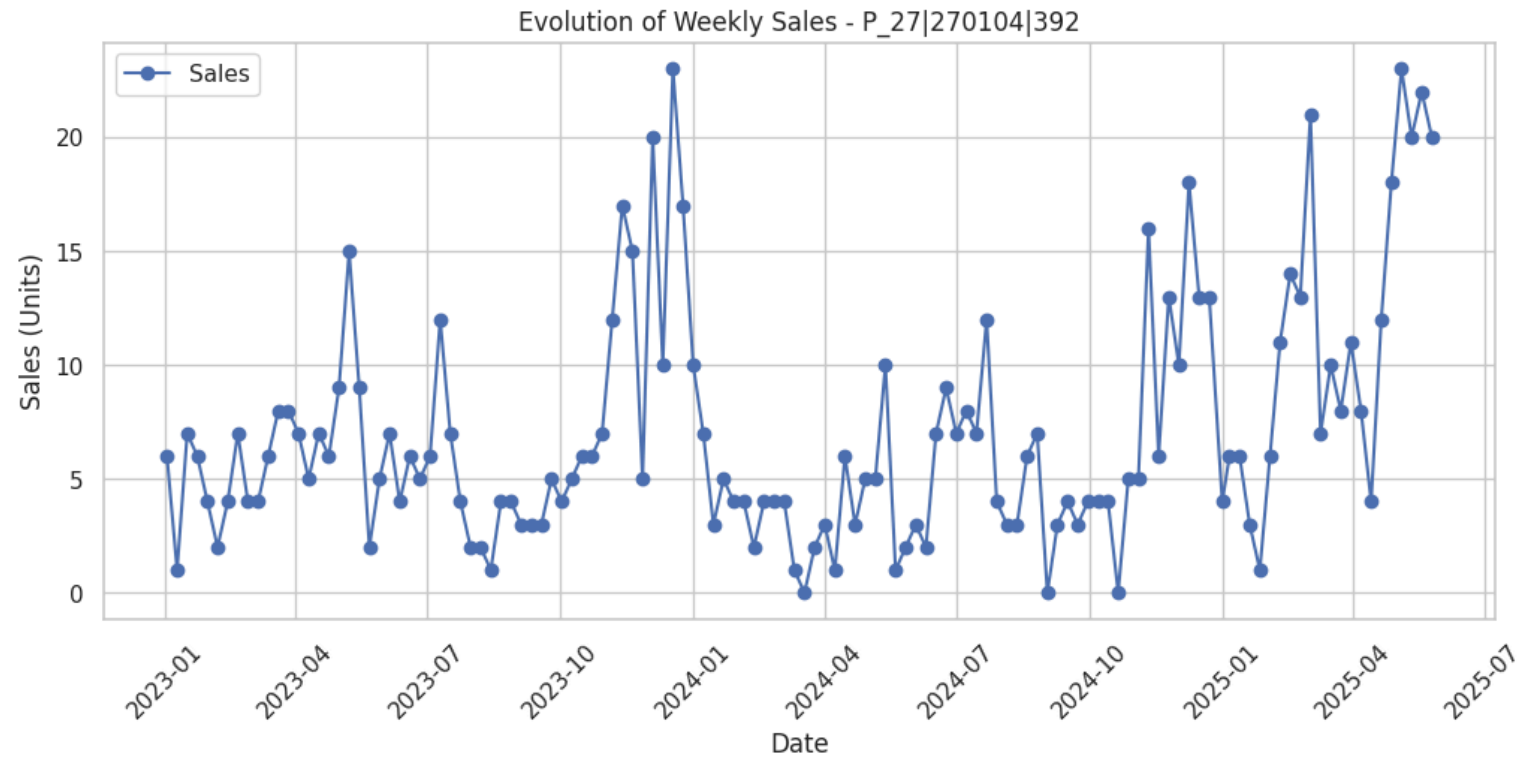
0.5916778836490239

```
Graph_sale('P_3|30103|614')
```



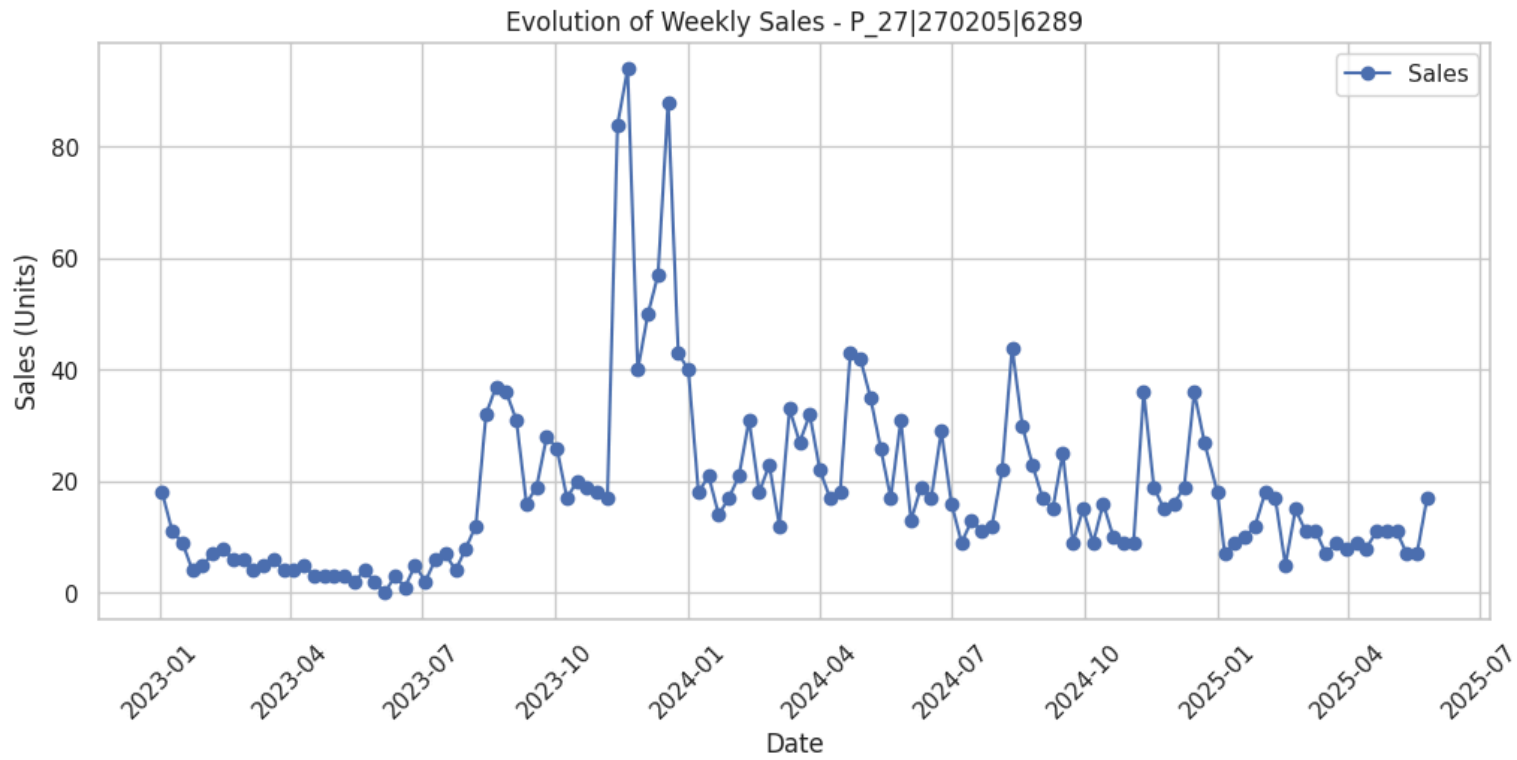
0.4764077671562434

Graph_sale('P_27|270104|392')

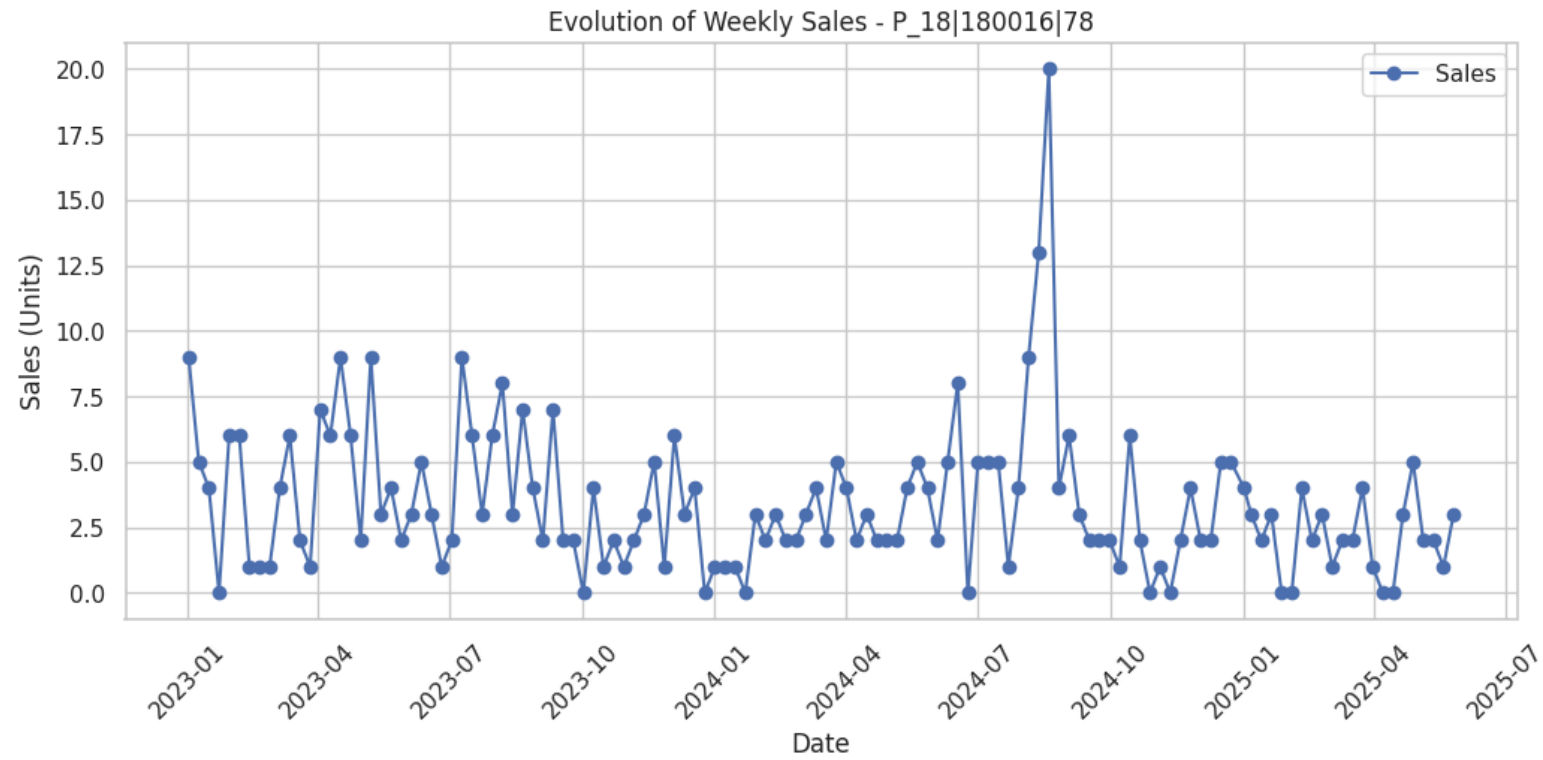


0.6307156221795849

Graph_sale('P_27|270205|6289')



```
Graph_sale('P_18|180016|78')
```

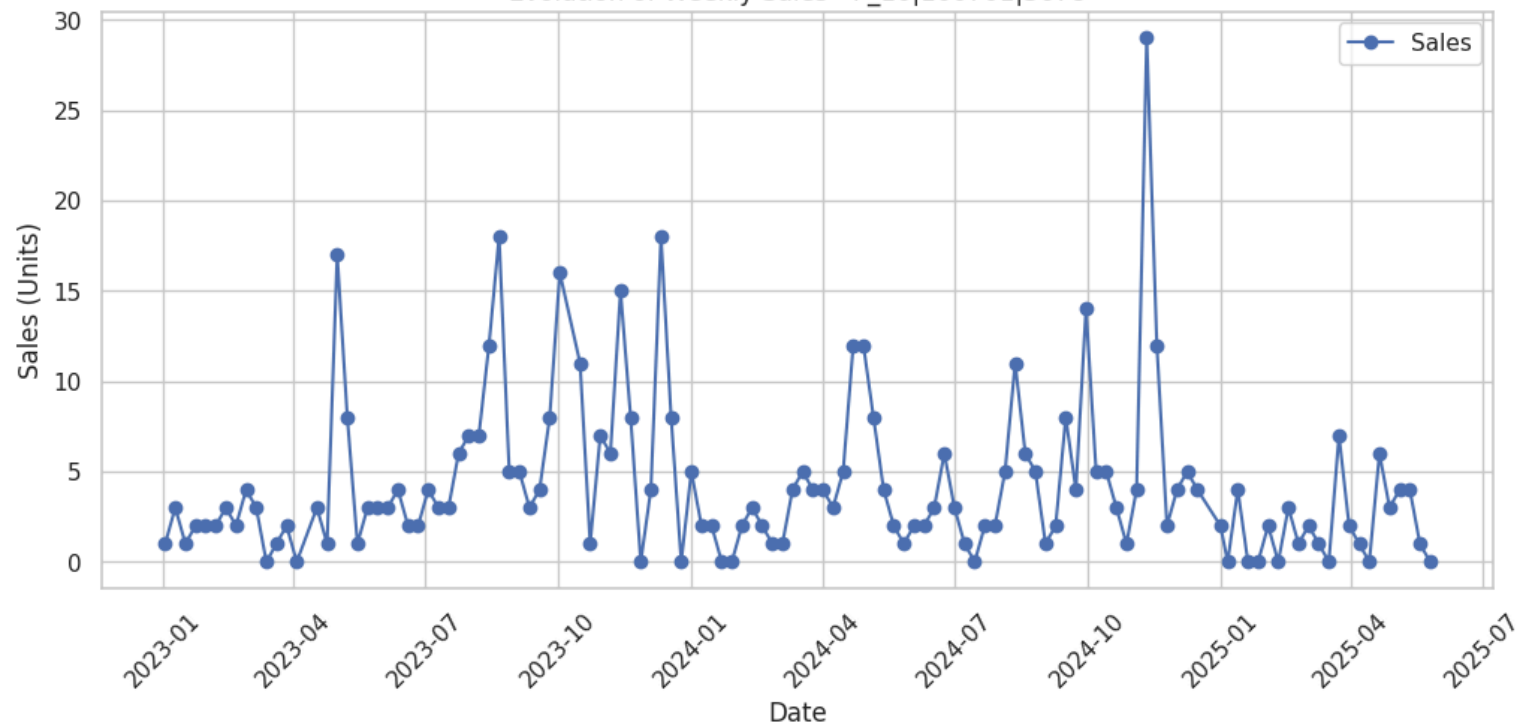


0.35184277161134386

Graph_sale('P_10|100701|3678')



Evolution of Weekly Sales - P_10|100701|3678



```
data2= data1[(data1['Department']==3) & (data1['Category']==30103) & (data1['Brand']==614)]
```

```
data2
```

[Afficher la sortie masquée](#)

Étapes suivantes :

[Générer du code avec data2](#)[Afficher les graphiques recommandés](#)[New interactive sheet](#)

```
data2['Date']= pd.to_datetime(data2['Date'])
```

```
data2= data2.sort_values('Date')
```

```
data2
```

[Afficher la sortie masquée](#)

Étapes suivantes :

[Générer du code avec data2](#)[Afficher les graphiques recommandés](#)[New interactive sheet](#)

ARIMA

```

def smape(y_true, y_pred):
    return 100 * np.mean(2 * np.abs(y_pred - y_true) / (np.abs(y_true) + np.abs(y_pred)))

def plot_results(train, test, pred_test, forecast_future, product_id):
    fig, axs = plt.subplots(3, 1, figsize=(12, 15))

    axs[0].plot(train.index, train, label="Train")
    axs[0].plot(test.index, test, label="Test", color="orange")
    axs[0].plot(test.index, pred_test, label="Pred test", color="green")
    axs[0].legend()
    axs[0].set_title(f"Historique + prédiction test ({product_id})")

    axs[1].plot(test.index, test, label="Test", color="orange")
    axs[1].plot(test.index, pred_test, label="Pred test", color="green")
    axs[1].legend()
    axs[1].set_title("Zoom période test")

    axs[2].plot(pd.concat([train, test]).index, pd.concat([train, test]), label="Historique")
    axs[2].plot(forecast_future.index, forecast_future, label="Prévision future", color="red")
    axs[2].legend()
    axs[2].set_title("Prévisions futures jusqu'à fin 2026")

    plt.tight_layout()
    plt.show()

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, mean_absolute_percentage_error
import numpy as np

def smape(y_true, y_pred):
    return 100 * np.mean(2 * np.abs(y_pred - y_true) / (np.abs(y_true) + np.abs(y_pred)))

def run_arima_for_product(df, dept, cat, brand):
    product_id = f"Dept={dept} | Cat={cat} | Brand={brand}"

    prod_df = df[(df["Department"] == dept) &
                  (df["Category"] == cat) &
                  (df["Brand"] == brand)].copy()

    prod_df["Date"] = pd.to_datetime(prod_df["Date"])
    prod_df = prod_df.sort_values("Date").set_index("Date")
    prod_df = prod_df.resample("W")["Sales (Units)"].sum()

    y = np.log1p(prod_df)

```

```

split_idx = int(len(y) * 0.8)
train, test = y.iloc[:split_idx], y.iloc[split_idx:]

model = ARIMA(train, order=(2,1,2))
model_fit = model.fit()
pred_test = model_fit.forecast(steps=len(test))

# Transformation inverse
y_test_actual = np.expm1(test)
y_test_pred = np.expm1(pred_test)

# Évaluation
mae_val = mean_absolute_error(y_test_actual, y_test_pred)
rmse_val = np.sqrt(mean_squared_error(y_test_actual, y_test_pred))
r2_val = r2_score(y_test_actual, y_test_pred)
mape_val = mean_absolute_percentage_error(y_test_actual, y_test_pred) * 100
smape_val = smape(y_test_actual, y_test_pred)

print(f"\n=== {product_id} ===")
print(f"MAE   : {mae_val:.2f}")
print(f"RMSE  : {rmse_val:.2f}")
print(f"R²    : {r2_val:.4f}")
print(f"MAPE   : {mape_val:.2f}%")
print(f"sMAPE : {smape_val:.2f}%")

model_full = ARIMA(y, order=(2,1,2))
model_full_fit = model_full.fit()

last_date = y.index.max()
future_weeks = pd.date_range(last_date + pd.Timedelta(weeks=1), "2026-12-31", freq="W")
forecast_future = pd.Series(model_full_fit.forecast(steps=len(future_weeks)), index=future_weeks)

plot_results(train, test, pred_test, forecast_future, product_id)

return pred_test, forecast_future

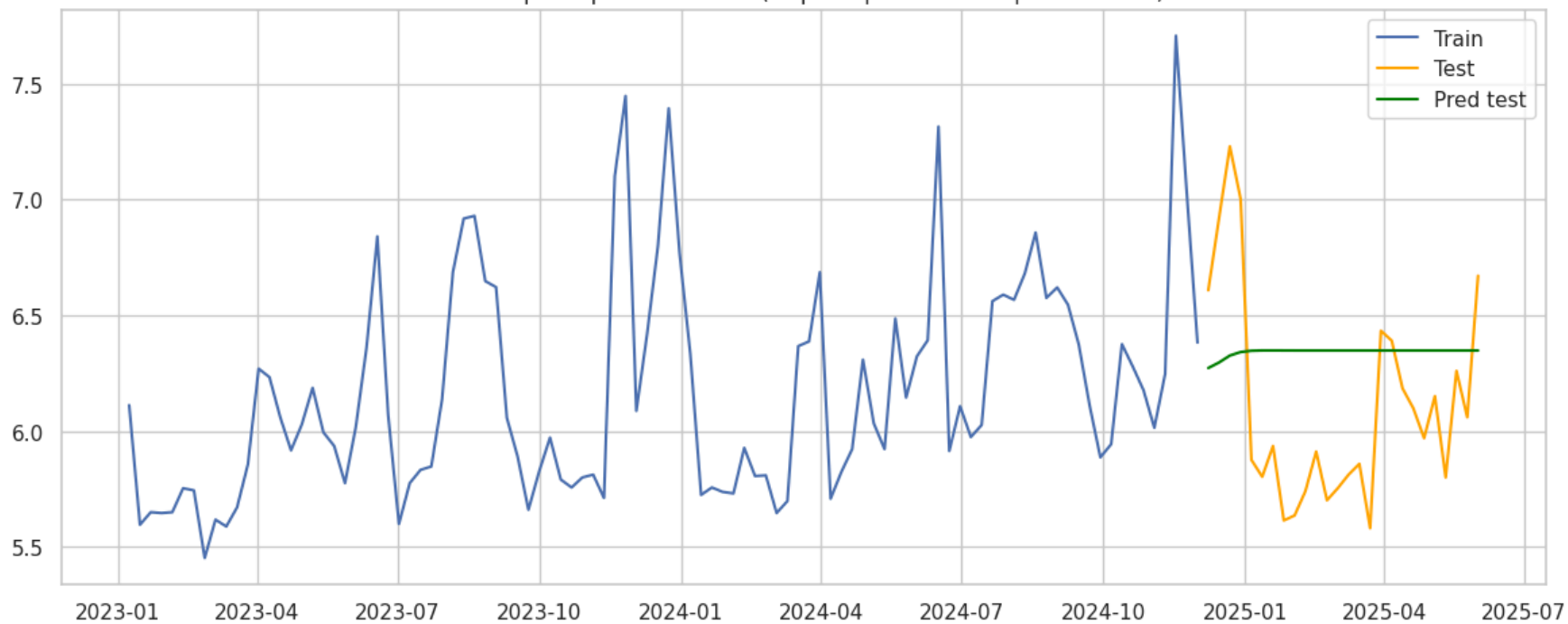
pred_test, forecast_future = run_arma_for_product(
    data1,
    dept=3,
    cat=30103 ,
    brand=614
)

```

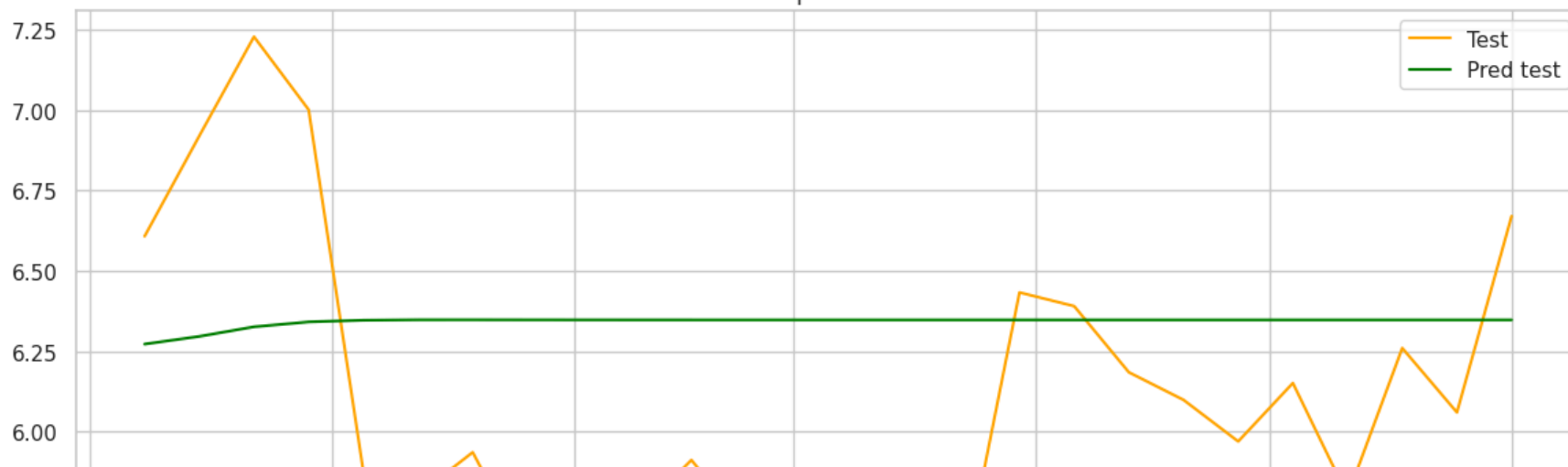


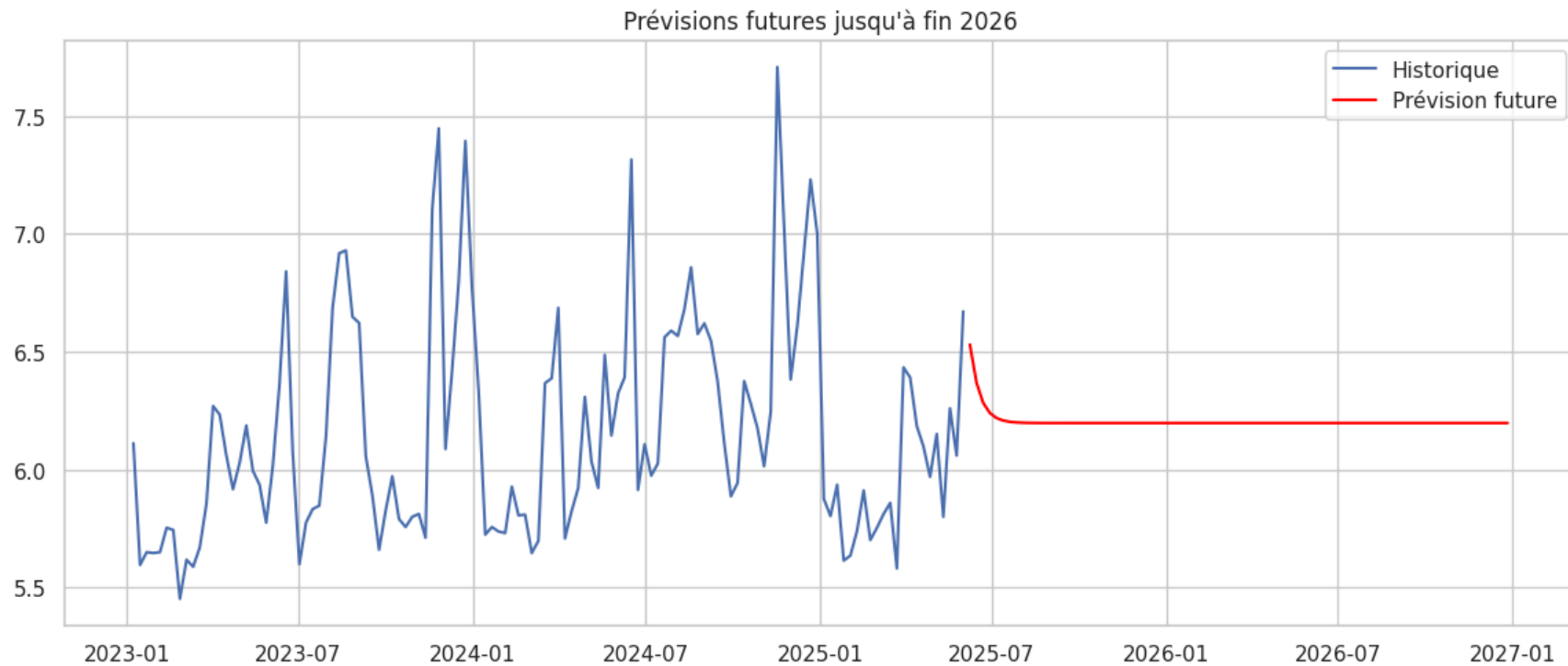
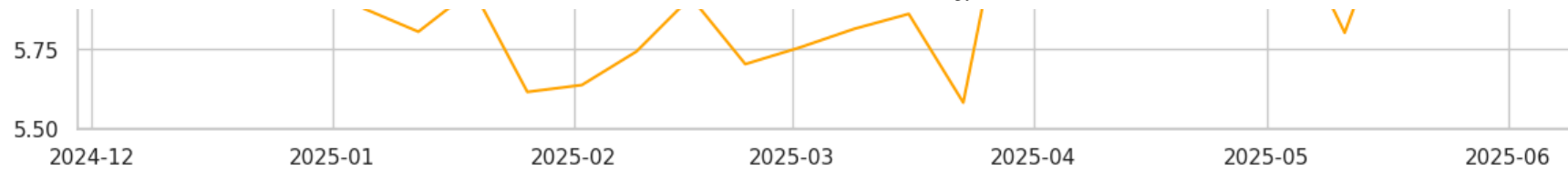
=== Dept=3 | Cat=30103 | Brand=614 ===
MAE : 240.22
RMSE : 289.68
R² : -0.0865
MAPE : 54.35%
sMAPE : 44.36%

Historique + prédiction test (Dept=3 | Cat=30103 | Brand=614)



Zoom période test



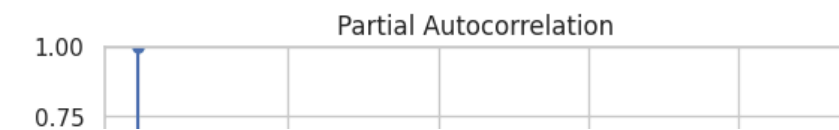
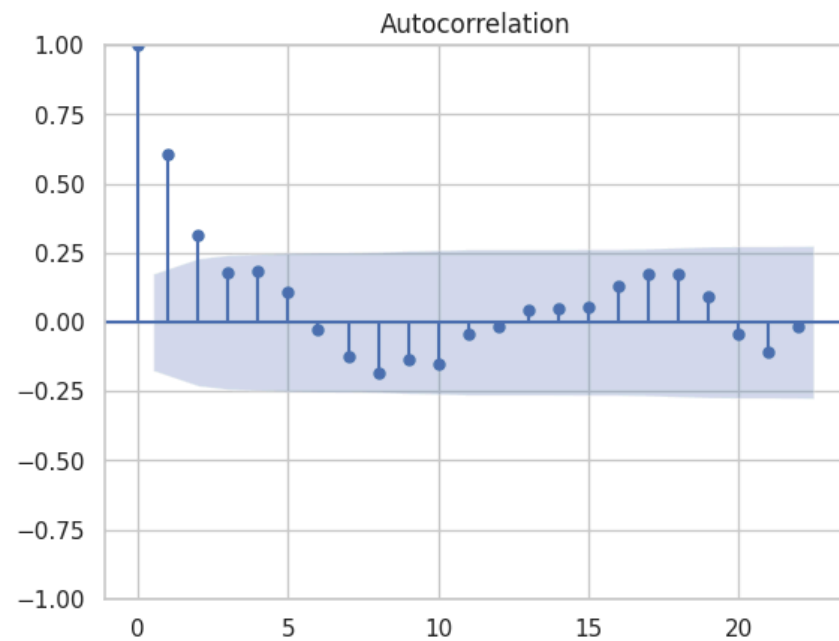
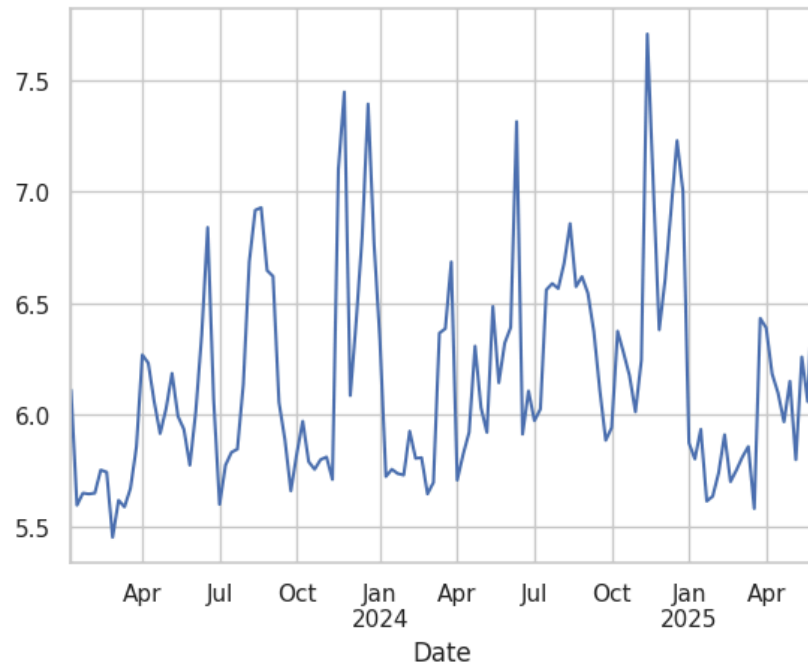


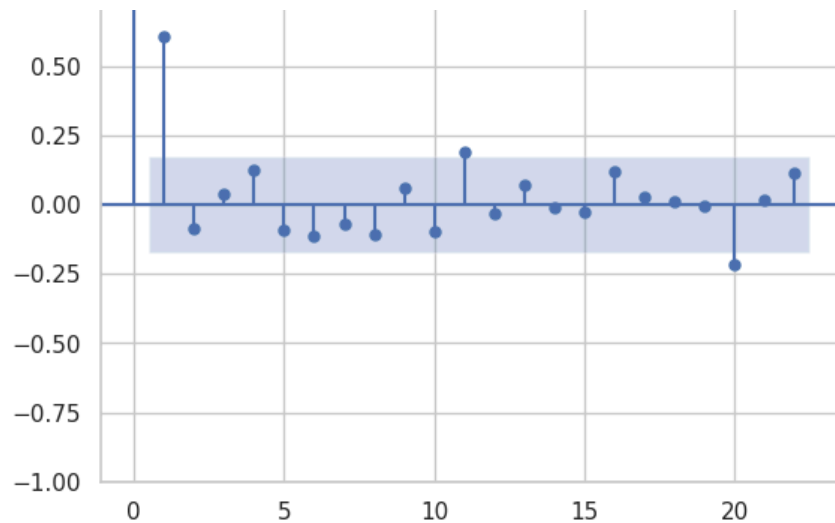
```
prod_df = data1[(data1["Department"] == 3) &
                 (data1["Category"] == 30103) &
                 (data1["Brand"] == 614)].copy()

prod_df["Date"] = pd.to_datetime(prod_df["Date"])
prod_df = prod_df.sort_values("Date").set_index("Date")
prod_df = prod_df.resample("W")["Sales (Units)"].sum()

# Apply log-transform
y = np.log1p(prod_df)

y.plot()
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(y); plot_pacf(y)
plt.show()
```





SARIMA

```
def smape(y_true, y_pred):
    return 100 * np.mean(2 * np.abs(y_pred - y_true) / (np.abs(y_true) + np.abs(y_pred)))

def plot_results(train, test, pred_test, forecast_future, product_id):
    fig, axs = plt.subplots(3, 1, figsize=(12, 15))

    axs[0].plot(train.index, np.expm1(train), label="Train")
    axs[0].plot(test.index, np.expm1(test), label="Test", color="orange")
    axs[0].plot(test.index, np.expm1(pred_test).clip(0), label="Pred test", color="green")
    axs[0].legend()
    axs[0].set_title(f"Historique + prédiction test ({product_id})")

    axs[1].plot(test.index, np.expm1(test), label="Test", color="orange")
    axs[1].plot(test.index, np.expm1(pred_test).clip(0), label="Pred test", color="green")
    axs[1].legend()
    axs[1].set_title("Zoom période test")

    axs[2].plot(pd.concat([train, test]).index, np.expm1(pd.concat([train, test])), label="Historique")
    axs[2].plot(forecast_future.index, np.expm1(forecast_future).clip(0), label="Prévision future", color="red")
    axs[2].legend()
    axs[2].set_title("Prévisions futures jusqu'à fin 2026")

    plt.tight_layout()
    plt.show()
```

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

def run_sarima_for_product(df, dept, cat, brand, seasonal_period=52, epsilon=1e-3):
    product_id = f"Dept={dept} | Cat={cat} | Brand={brand}"

    prod_df = df[(df["Department"] == dept) &
                  (df["Category"] == cat) &
                  (df["Brand"] == brand)].copy()

    prod_df["Date"] = pd.to_datetime(prod_df["Date"])
    prod_df = prod_df.sort_values("Date").set_index("Date")
    prod_df = prod_df.resample("W")["Sales (Units)"].sum()

    y = np.log1p(prod_df)

    # ACF/PACF
    plot_acf(y)
    plot_pacf(y)
    plt.show()

    # Split train/test (80/20)
    split_idx = int(len(y) * 0.8)
    train, test = y.iloc[:split_idx], y.iloc[split_idx:]

    # SARIMAX
    model = SARIMAX(train, order=(2,0,1), seasonal_order=(2,0,1,seasonal_period))
    model_fit = model.fit(dispatch=False)
    pred_test = model_fit.get_forecast(steps=len(test)).predicted_mean

    # Transformation inverse
    y_test_actual = np.expml(test) + epsilon
    y_test_pred = np.expml(pred_test).clip(0) + epsilon

    # Évaluation
    mae_val = mean_absolute_error(y_test_actual, y_test_pred)
    rmse_val = np.sqrt(mean_squared_error(y_test_actual, y_test_pred))
    r2_val = r2_score(y_test_actual, y_test_pred)
    mape_val = mean_absolute_percentage_error(y_test_actual, y_test_pred) * 100
    smape_val = smape(y_test_actual, y_test_pred)

    print(f"\n=== {product_id} ===")
    print(f"MAE : {mae_val:.2f}")
    print(f"RMSE : {rmse_val:.2f}")
    print(f"R² : {r2_val:.4f}")
    print(f"MAPE : {mape_val:.2f}%")
    print(f"sMAPE: {smape_val:.2f}%")

    # Prévisions futures
    model_full = SARIMAX(y, order=(2,1,2), seasonal_order=(1,1,1,seasonal_period))
    model_full_fit = model_full.fit(dispatch=False)

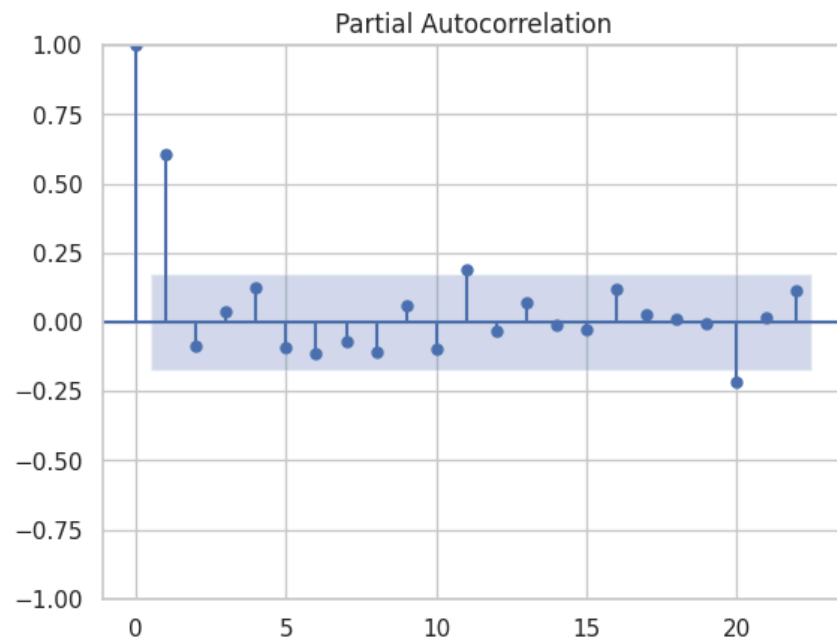
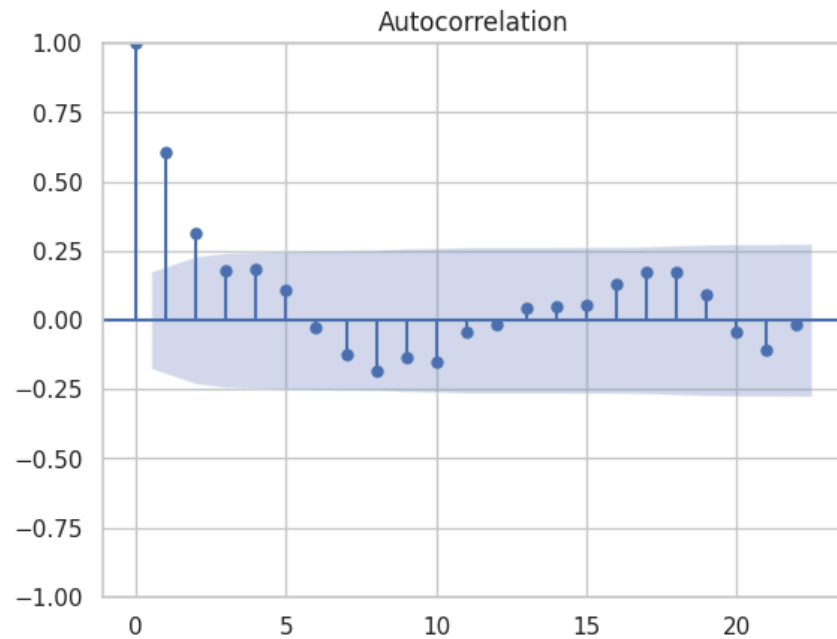
```

```
last_date = y.index.max()
future_weeks = pd.date_range(last_date + pd.Timedelta(weeks=1), "2026-12-31", freq="W")
forecast_future = pd.Series(model_full_fit.get_forecast(steps=len(future_weeks)).predicted_mean,
                             index=future_weeks)
forecast_future = forecast_future.clip(0)

plot_results(train, test, pred_test, forecast_future, product_id)

return pred_test, forecast_future

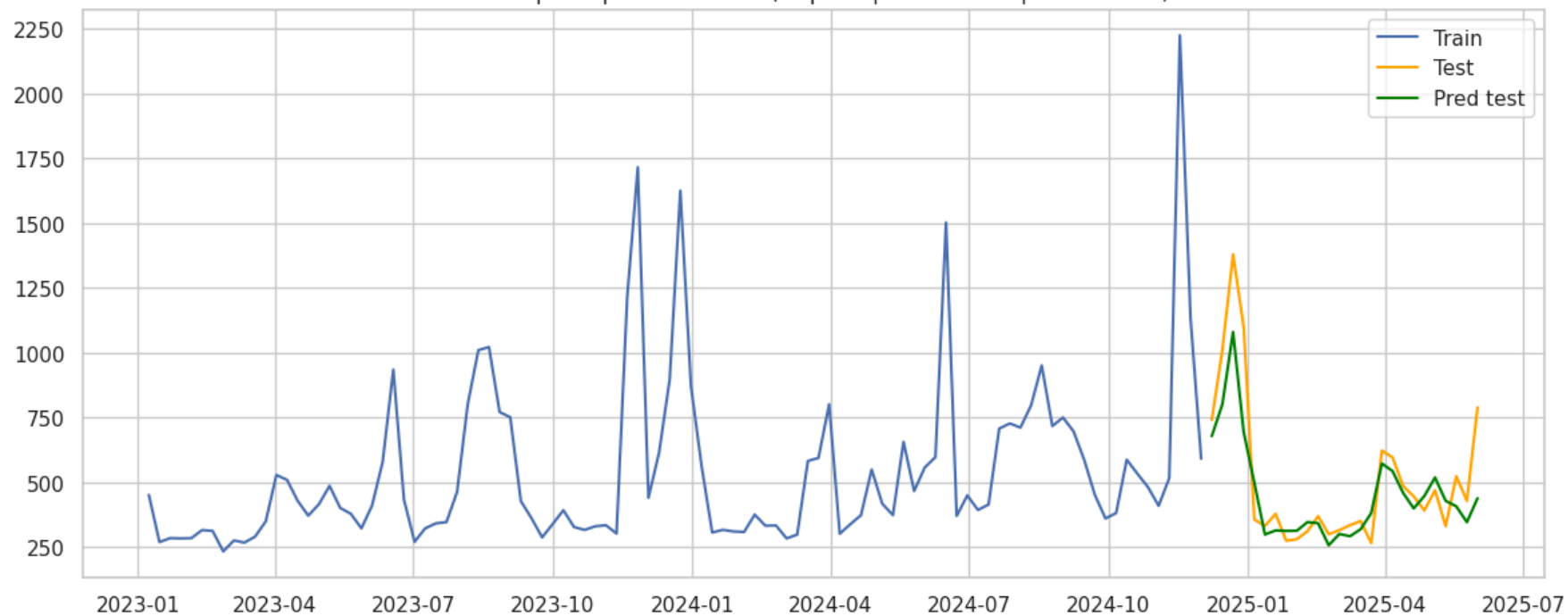
pred_test, forecast_future = run_sarima_for_product(
    data1,
    dept=3,
    cat=30103 ,
    brand=614
)
```



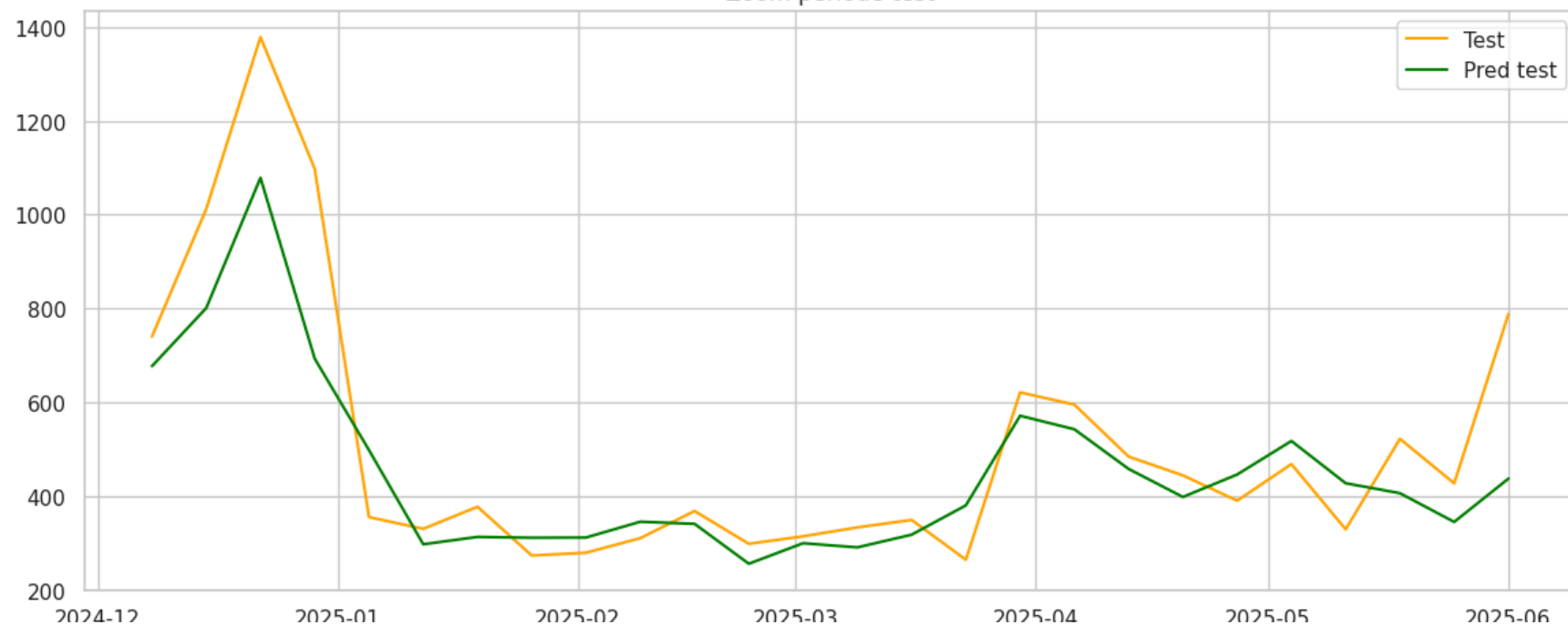
```
=== Dept=3 | Cat=30103 | Brand=614 ===  
MAE : 97.20  
RMSE : 141.11  
R2 : 0.7422  
MAPE : 17.61%
```

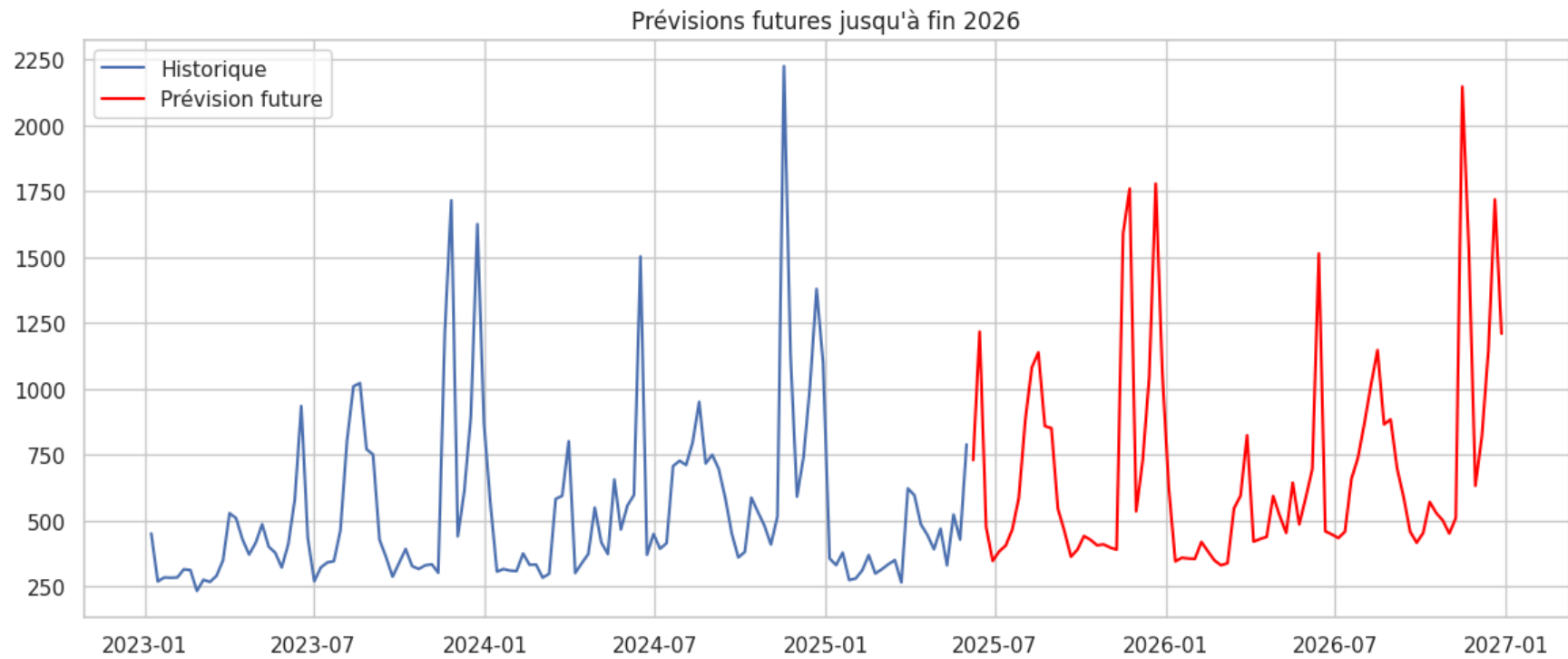
sMAPE: 18.21%

Historique + prédiction test (Dept=3 | Cat=30103 | Brand=614)



Zoom période test





SARIMAX

Double-cliquez (ou appuyez sur Entrée) pour modifier

```
def smape(y_true, y_pred):
    """Calcul du symmetric MAPE"""
    return 100 * np.mean(2 * np.abs(y_pred - y_true) / (np.abs(y_true) + np.abs(y_pred)))

def plot_results(train, test, pred_test, forecast_future, product_id):
    """Affichage des résultats"""
    fig, axs = plt.subplots(3, 1, figsize=(12, 15))

    axs[0].plot(train.index, train, label="Train")
    axs[0].plot(test.index, test, label="Test", color="orange")
    axs[0].plot(test.index, pred_test, label="Pred test", color="green")
    axs[0].legend()
    axs[0].set_title(f"Historique + prédiction test ({product_id})")

    axs[1].plot(test.index, test, label="Test", color="orange")
    axs[1].plot(test.index, pred_test, label="Pred test", color="green")
    axs[1].legend()
    axs[1].set_title("Zoom période test")

    axs[2].plot(pd.concat([train, test]).index, pd.concat([train, test]), label="Historique")
    axs[2].plot(forecast_future.index, forecast_future, label="Prévision future", color="red")
    axs[2].legend()
    axs[2].set_title("Prévisions futures jusqu'à fin 2026")

    plt.tight_layout()
    plt.show()

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def run_sarimax_product(df, dept, cat, brand, seasonal_period=52):
    product_id = f"Dept={dept} | Cat={cat} | Brand={brand}"

    prod_df = df[(df["Department"]==dept) &
                  (df["Category"]==cat) &
                  (df["Brand"]==brand)].copy()

    prod_df["Date"] = pd.to_datetime(prod_df["Date"])
    prod_df = prod_df.sort_values("Date").set_index("Date")
    prod_df = prod_df.resample("W")["Sales (Units)"].sum()

    y = prod_df
    y = y.clip(lower=0)
```

```

plot_acf(y)
plot_pacf(y)
plt.show()

# Split train/test (80/20)
split_idx = int(len(y) * 0.8)
train, test = y.iloc[:split_idx], y.iloc[split_idx:]

# SARIMAX
model = SARIMAX(train, order=(1,1,1), seasonal_order=(1,1,1,seasonal_period))
model_fit = model.fit(dispatch=False)

pred_test = model_fit.get_forecast(steps=len(test)).predicted_mean
pred_test = pred_test.clip(lower=0)

# Evaluation
mape_val = mean_absolute_percentage_error(test, pred_test) * 100
smape_val = smape(test, pred_test)
mae_val = mean_absolute_error(test, pred_test)
rmse_val = np.sqrt(mean_squared_error(test, pred_test))
r2_val = r2_score(test, pred_test)

print(f"\n=== {product_id} ===")
print(f"MAE : {mae_val:.2f}")
print(f"RMSE: {rmse_val:.2f}")
print(f"R² : {r2_val:.4f}")
print(f"MAPE : {mape_val:.2f}%")
print(f"sMAPE: {smape_val:.2f}%")

# Modèle complet pour prévisions futures
model_full = SARIMAX(y, order=(2,1,2), seasonal_order=(1,1,1,seasonal_period))
model_full_fit = model_full.fit(dispatch=False)

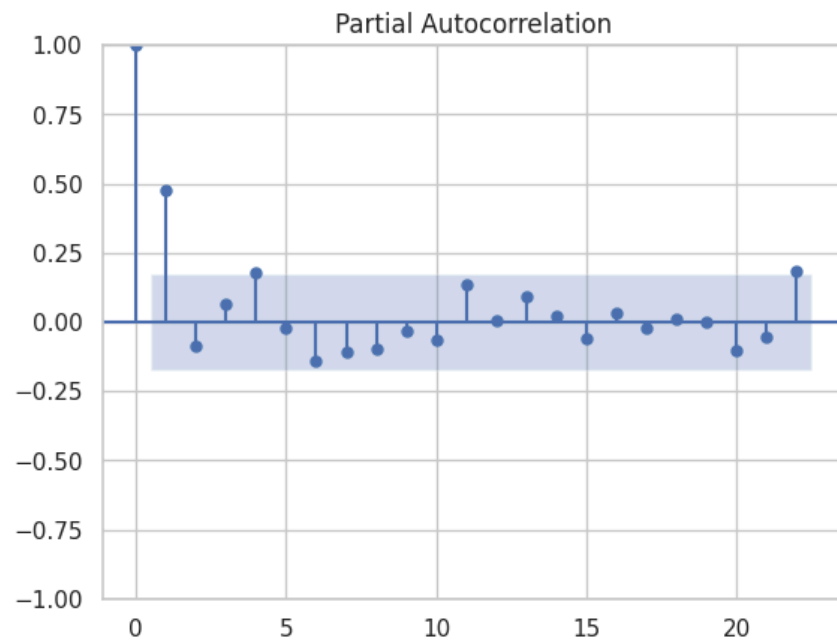
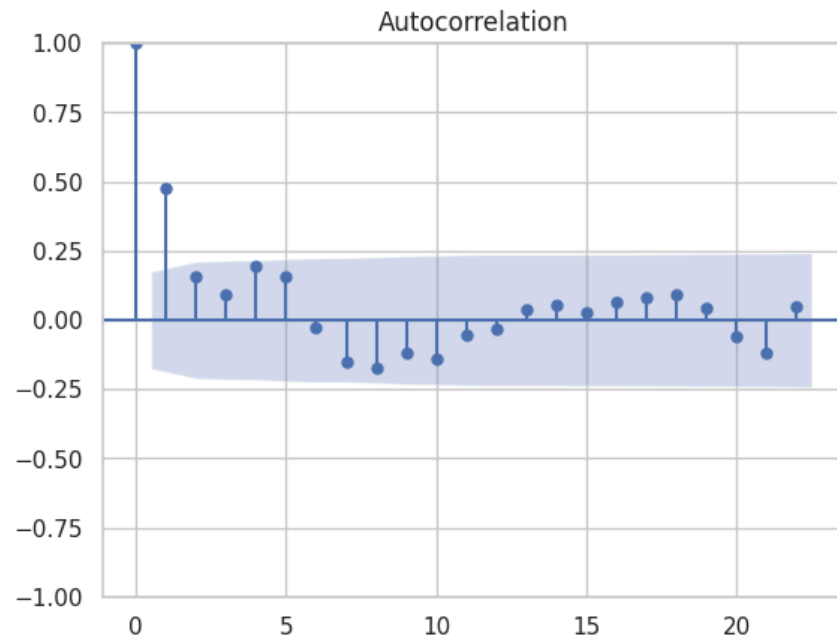
last_date = y.index.max()
future_weeks = pd.date_range(last_date + pd.Timedelta(weeks=1), "2026-12-31", freq="W")
forecast_future = pd.Series(model_full_fit.get_forecast(steps=len(future_weeks)).predicted_mean,
                             index=future_weeks)
forecast_future = forecast_future.clip(lower=0)

plot_results(train, test, pred_test, forecast_future, product_id)

return pred_test, forecast_future

pred_test, forecast_future = run_sarimax_product(
    data1,
    dept=3,
    cat=30103 ,
    brand=614
)

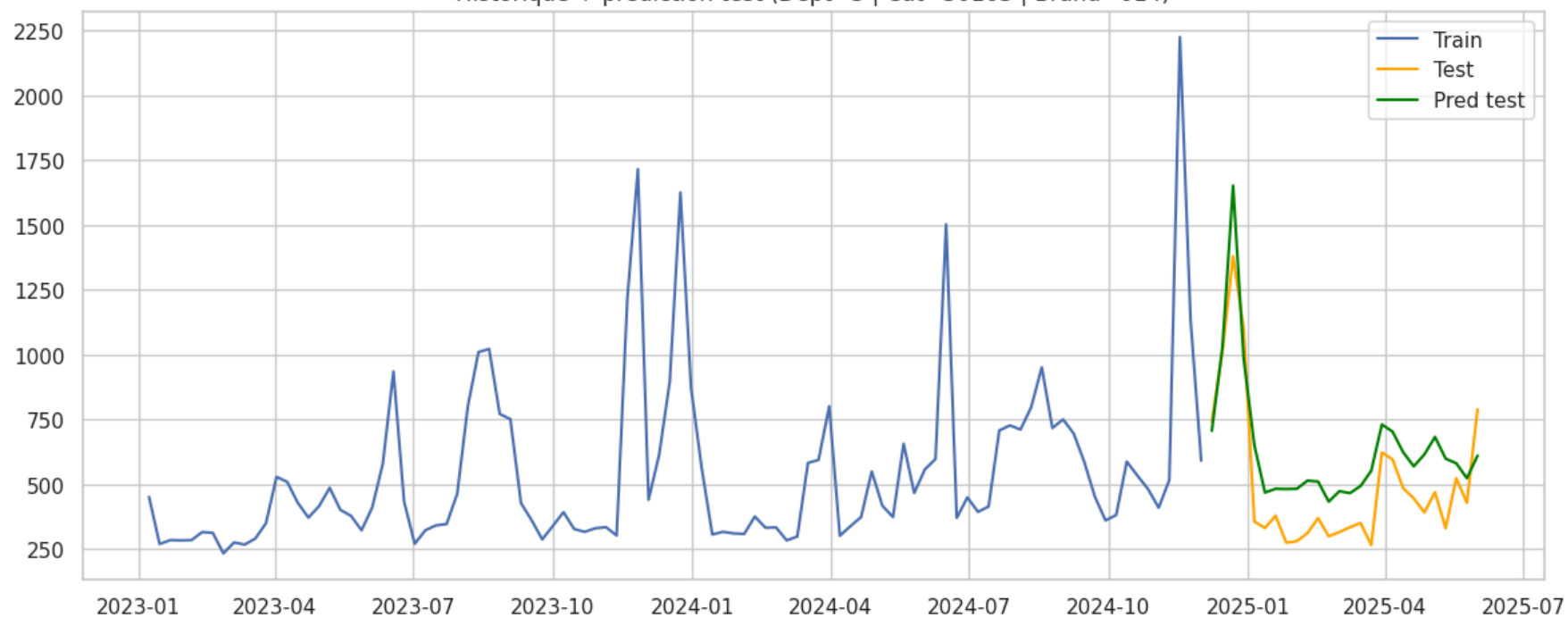
```



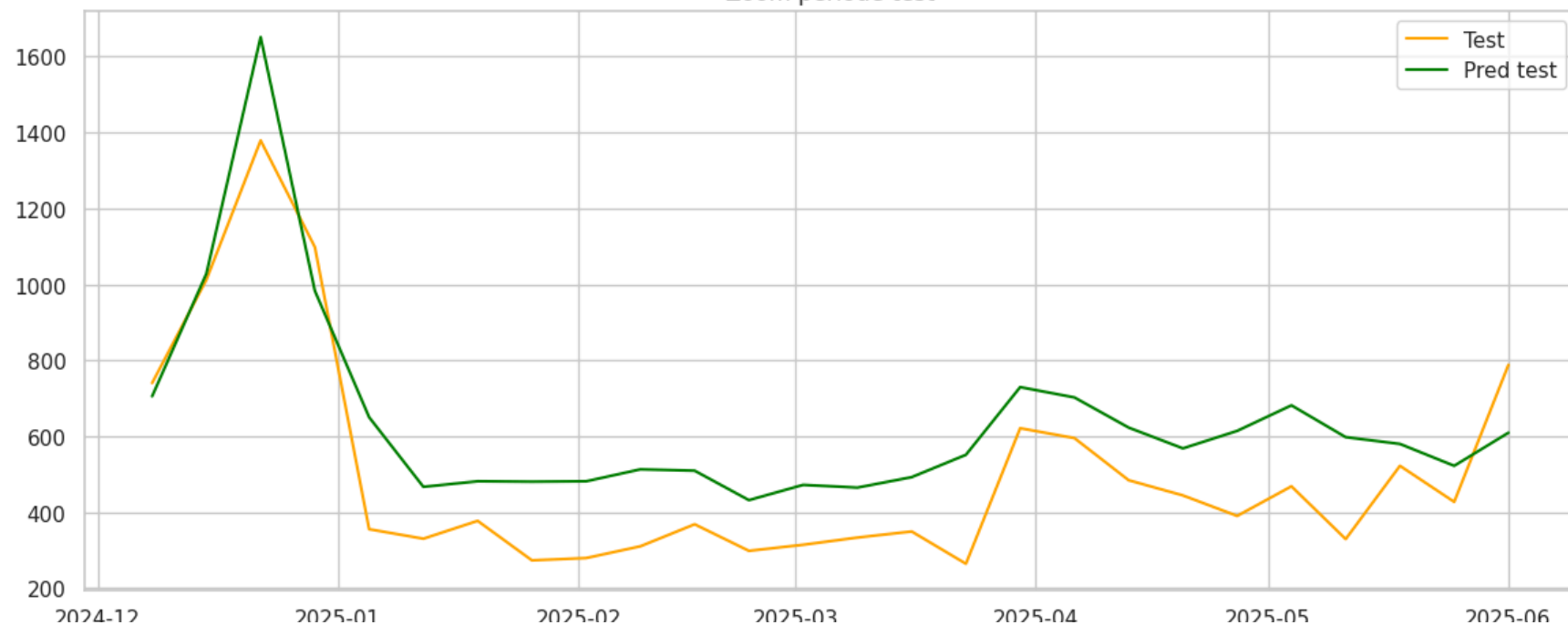
```
=== Dept=3 | Cat=30103 | Brand=614 ===  
MAE : 157.53  
RMSE: 173.54  
R2 : 0.6101  
MAPE : 40.68%
```

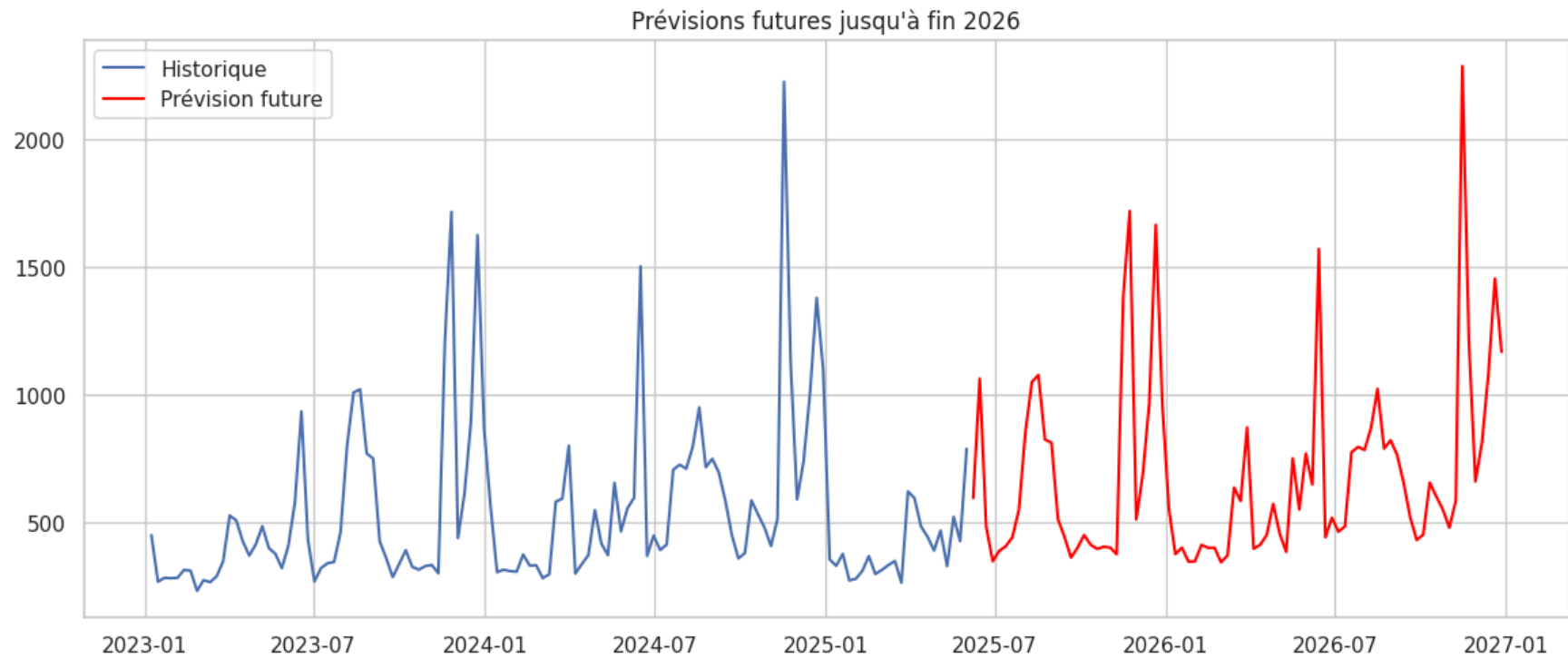
sMAPE: 32.10%

Historique + prédiction test (Dept=3 | Cat=30103 | Brand=614)



Zoom période test





Ce texte est au format code

Model Prophet

```
def prophet_prediction(data1, department, category, brand, n_weeks=84):

    df = data1[
        (data1['Department'] == department) &
        (data1['Category'] == category) &
        (data1['Brand'] == brand)
    ].copy()

    if df.empty:
        print("Aucune donnée trouvée pour ce produit.")
        return None

    df['Date'] = pd.to_datetime(df['Date'])
    df = df.groupby('Date')['Sales (Units)'].sum().reset_index()
    df = df.rename(columns={'Date': 'ds', 'Sales (Units)': 'y'})
    df = df.sort_values('ds')

    if df['y'].sum() == 0 or len(df) < 20:
        print("Pas assez de données valides pour ce produit.")
        return None

    model = Prophet(weekly_seasonality=True, yearly_seasonality=True, daily_seasonality=False)
    model.fit(df)

    future = model.make_future_dataframe(periods=n_weeks, freq='W')
    forecast = model.predict(future)

    forecast = forecast.rename(columns={'yhat': 'Predicted_Sales'})

    df_merge = df.merge(forecast[['ds', 'Predicted_Sales']], on='ds', how='left')

    plt.figure(figsize=(12,6))
    plt.plot(df_merge['ds'], df_merge['y'], label='Ventes réelles', color='blue')
    plt.plot(forecast['ds'], forecast['Predicted_Sales'], label='Prévisions Prophet', color='orange')
    plt.title(f"Prévision Prophet - Dept: {department}, Cat: {category}, Brand: {brand}")
    plt.xlabel("Date")
    plt.ylabel("Ventes hebdomadaires")
    plt.legend()
```

```
plt.grid(True)
plt.tight_layout()
plt.show()

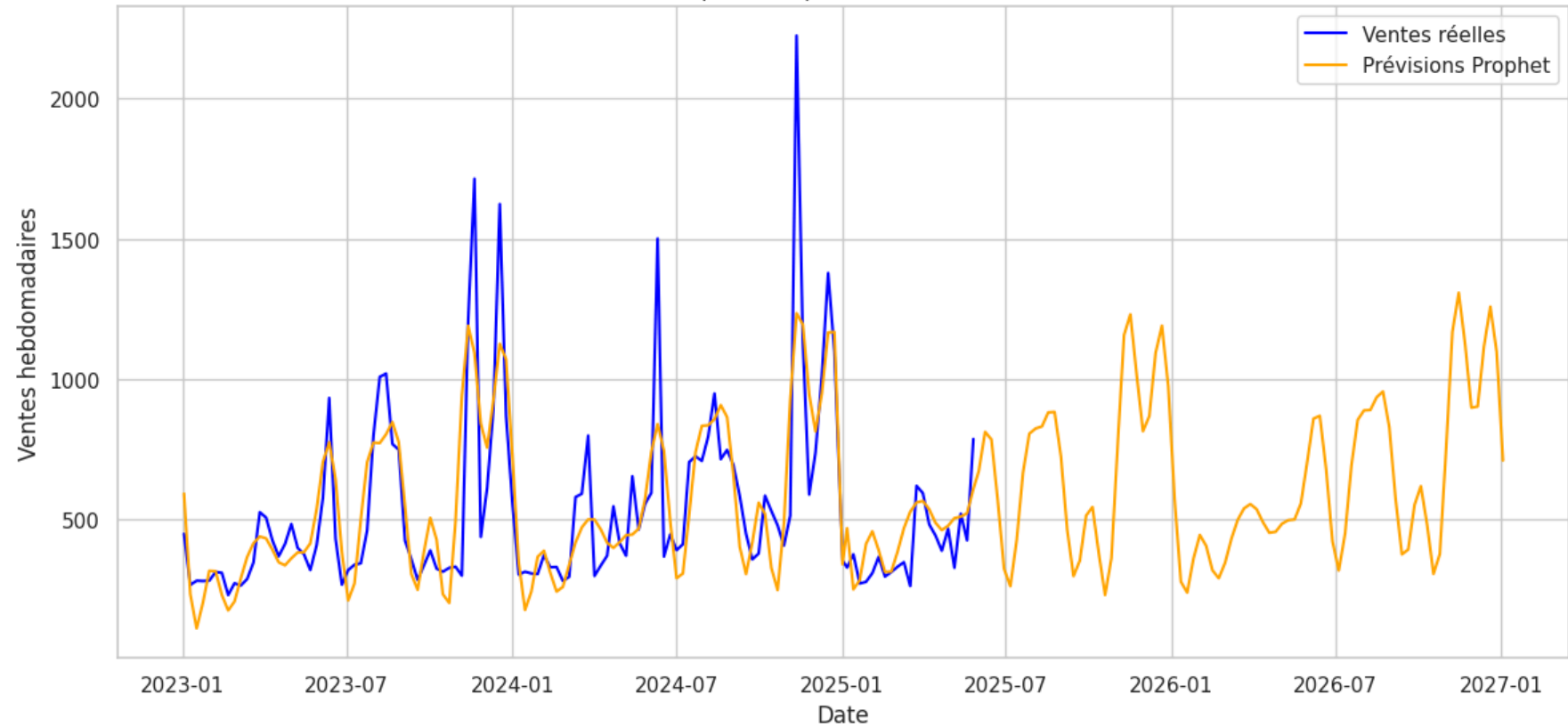
return forecast[['ds', 'Predicted_Sales', 'yhat_lower', 'yhat_upper']]


resultats = prophet_prediction(data1, department=3, category=30103 , brand=614)

if resultats is not None:
    print("\n--- Valeurs prédites par semaine ---")
    for i, row in resultats.iterrows():
        date = row['ds'].strftime('%Y-%m-%d')
        Predicted_Sales = round(row['Predicted_Sales'], 2)
        yhat_lower = round(row['yhat_lower'], 2)
        yhat_upper = round(row['yhat_upper'], 2)
        print(f"Semaine du {date} → Prédiction: {max(Predicted_Sales,0)} (min: {max(yhat_lower,0)}, max: {yhat_upper})")
```



Prévision Prophet - Dept: 3, Cat: 30103, Brand: 614



--- Valeurs prédites par semaine ---

Semaine du 2023-01-02 → Prédiction: 593.35 (min: 346.86, max: 844.78)
 Semaine du 2023-01-09 → Prédiction: 235.23 (min: 0, max: 493.4)
 Semaine du 2023-01-16 → Prédiction: 113.07 (min: 0, max: 361.15)
 Semaine du 2023-01-23 → Prédiction: 204.73 (min: 0, max: 449.06)
 Semaine du 2023-01-30 → Prédiction: 318.39 (min: 80.9, max: 539.49)
 Semaine du 2023-02-06 → Prédiction: 317.07 (min: 62.2, max: 559.72)
 Semaine du 2023-02-13 → Prédiction: 231.94 (min: 0, max: 472.38)
 Semaine du 2023-02-20 → Prédiction: 177.15 (min: 0, max: 440.86)
 Semaine du 2023-02-27 → Prédiction: 208.93 (min: 0, max: 454.31)
 Semaine du 2023-03-06 → Prédiction: 291.46 (min: 38.77, max: 547.32)
 Semaine du 2023-03-13 → Prédiction: 367.49 (min: 127.4, max: 604.04)
 Semaine du 2023-03-20 → Prédiction: 416.42 (min: 179.43, max: 656.02)
 Semaine du 2023-03-27 → Prédiction: 440.49 (min: 190.05, max: 692.9)
 Semaine du 2023-04-03 → Prédiction: 433.71 (min: 204.42, max: 697.5)
 Semaine du 2023-04-10 → Prédiction: 393.7 (min: 158.79, max: 662.35)
 Semaine du 2023-04-17 → Prédiction: 348.54 (min: 109.16, max: 581.87)
 Semaine du 2023-04-24 → Prédiction: 337.86 (min: 99.03, max: 582.37)
 Semaine du 2023-05-01 → Prédiction: 362.85 (min: 106.42, max: 605.37)
 Semaine du 2023-05-08 → Prédiction: 383.67 (min: 135.19, max: 627.69)
 Semaine du 2023-05-15 → Prédiction: 384.36 (min: 130.95, max: 648.93)
 Semaine du 2023-05-22 → Prédiction: 416.21 (min: 190.07, max: 661.7)